

Where to Stop: Tile-Adaptive Early Exit in a Learned Image Decoder

Supplementary Material

Anonymous CVPR submission · Paper ID ****

A. Test settings, implementation and reproducibility

A.1. What this section covers

This section is the recipe and the protocol: the released codec the work starts from and the exact file it is, the test set, the conventions every decibel is measured under, every module with its shape and its parameter count, the two algorithms a reader would have to re-implement, the training configuration with a column naming where each choice was measured, the hardware, the wall clock, the seeds and what varies between two runs of one command. The sections after it account for compute (B), derive the allocation rule and its bounds (C), report the complete results (D), describe the quality budget and its working range (E) and measure the decoder-side router (F). Numbers are not repeated across sections; where a quantity belongs to a later one, this section names it and stops. One fact governs the rest and is stated here rather than buried: the checkpoint every headline number comes from has seen the training set exactly once, and A.11 reports how much the numbers move with a second pass.

A.2. Test settings: the released codec, and what is frozen

There is no traditional-codec invocation in this work and none is needed. The axis measured here is decoder-side compute against a fixed learned decoder, on a bitstream that decoder produced, so the anchor is a checkpoint rather than a command line, and the released codec's own standing against VTM is published with it. That checkpoint is identified by content and not by path: results/dmc_ld_recon_audit.json records ~/DCVC/checkpoints/cvpr2026_image.pth.tar with sha256 b3b900de23f30e4fc437010ddffe6a54 and the note that the file is a bare state dict carrying no epoch, no optimiser and no step, so path plus hash is the whole of its provenance.

The codec is imported rather than reimplemented: flexuf/model.py sets DCVC_ROOT to the DCVC checkout and imports src.models.image_model.DMCI from it, so the entropy model, the hyperprior, the quantisation-step tables and the 64 quality indices are the released code running unmodified. Our decoder is those weights rearranged: an opening upsampling block, 12 DepthConvBlocks at C = 384 channels and a head that pixel-shuffles back to RGB, with the 12 blocks grouped into K = 6 exits of b = 2 blocks each and the first j = 2 groups running once over the whole frame before the decode splits into tiles. Exits 0 and 1 therefore lie below the split and cannot be selected; the four reachable exits are 2 to 5.

The encoder is frozen for the whole of training, and that is asserted rather than assumed: scripts/signalled_curve.py compares our encoder state dict against the released one tensor by tensor and requires the largest absolute difference to be exactly zero before a single frame is measured. A trained decoder therefore consumes byte for byte the stream the released encoder produces, the bit rate is identical by construction, and the whole of the measured difference is decoder-side.

A.3. The test set, and what tiling does to it

class	sequences	resolution	padded to	tiles	added px
UVG	7	1920x1080	2048x1280	40	26.4%
MCL-JCV	30	1920x1080	2048x1280	40	26.4%
HEVC B	5	1920x1080	2048x1280	40	26.4%
HEVC E	3	1280x720	1280x768	15	6.7%
HEVC C	4	832x480	1024x512	8	31.3%
HEVC D	4	416x240	512x256	2	31.3%
all	53				

Table 77. The test set, and what tiling does to each resolution. Take from it that granularity is not constant across the set: a 1080p frame carries 40 tiles for the allocation to work with and a 416x240 frame carries two, and the small classes also pay the most padding. That single fact explains most of the per-class spread reported in section D.

Sources: results/per_class_RECIPES12.json for the class membership and tile counts, results/tile_definition.json for the padded sizes; both on runs/RECIPES12/ckpt_PAPER.pth.tar. The sequences are UVG [25], MCL-JCV [26] and HEVC classes B, C, D and E [9]; the full list of 53 names is recorded in the measured field of results/signalled_RECIPES12_ctc53.json, with not_measured empty.

Frames are the leading frames of each sequence, taken consecutively. The headline file uses two frames on each of the 53 sequences of Table 77, so 106 frames; the routed, rate-rank and partial signalling tables use one. Every results file records frames_per_seq and every table in this supplement states it. No frame is dropped and no sequence excluded: the not_measured field of every curve file is empty.

A.4. The measurement conventions

- **Colour.** Sources are YUV 4:2:0, 8 bit, converted to 4:4:4, divided by 255 and shifted by -0.5 for the network. PSNR is the released codec's own metric, $(6 \cdot Y + U + V)/8$, computed after converting the reconstruction back to 4:2:0 on the 0 to 255 scale. Measuring in 4:4:4 would compare against a chroma upsample of the source rather than the source, and would report a chroma PSNR no published DCVC-UF number could be compared with.
- **Padding.** Frames are padded by replication, bottom and right only, to a whole number of tiles; results/tile_definition.json records the site as the RGB frame before the encoder and the mode as "replicate, bottom and right only: F.pad(x, (0, pw, 0, ph))". The stock codec aligns to 16, which is enough for the transform and not for a tile grid. Rate is charged on the true pixel count and PSNR computed after cropping back, so padding can cost and cannot flatter, and it does cost: a tile containing replicated rows gives up about twice the quality of one that does not.
- **Decibels.** A budget is a loss against the released decoder on the same latent, averaged per frame and then over frames, which is the convention the released codec's own evaluation uses. Pooling all tiles into one mean squared error first is the other live convention and is not the same number; section E measures the gap at about a third of a 0.1 dB budget. No table mixes the two and every table says which it is on.
- **A second metric.** MS-SSIM is reported beside PSNR at the operating point, computed as Wang et al. 2003, 5 scales, 11-tap Gaussian sigma 1.5, valid region, luma plane in 4:2:0 on 0..255. The implementation is self-checked at start-up: identical inputs score 1.000000 and a 3x3 box blur scores 0.7797, both recorded in results/supp_opquality_PAPER.json.

- **The map is billed.** The exit map is entropy coded from a per-frame histogram and its cost added to the rate before any saving is computed: 79 to 95 bits per frame at the 0.1 dB budget, at most 9.09e-05 bpp. Every row records map_bits and the bpp it added.
- **The reference.** The released decoder re-expressed in the same ladder and selected by K, so a K = 12 run cannot be quoted against the K = 6 remap.

Two limits belong beside that list. Sweeping a multiplier reaches only the lower convex hull of the achievable set [28, 29], so a budget falling in a gap of the hull is met at the nearest reachable point; every row carries a reachability flag and a floor, and rows whose floor already exceeds the budget are reported as unreachable rather than dropped. And 0.1 dB is a reporting convention, not a perceptual threshold. Subjective work measures the smallest noticeable change in quantisation parameter or in a video quality metric, not in tenths of a decibel of PSNR, so nothing published licenses a claim that 0.1 dB is invisible; 0.3 dB and 0.5 dB are reported beside it so that the choice carries no weight.

A.5. The ladder, module by module

module	runs	in → out	params	% dec
upsample	frame	256 → 384, ×2	1,431,936	8.16
groups 0 to 1, stem	frame	384 → 384	4,154,880	29.81
groups 2 to 5, routable	tile	384 → 384	8,309,760	59.63
one DepthConvBlock	tile	384 → 384	1,038,720	7.45
adapters 0 to 3, FFN	tile, at its exit	384 → 1536 → 384	739,200	5.31
adapter 4, 1×1	tile, at its exit	384 → 384	147,840	1.06
seam repair, grid	after stitching	384 → 384	152,704	1.09
head, PixelShuffle 8	frame	384 → 192 → 3	335,232	2.40
router head (B)	tile	161 → 6	144,024	0.163
decoder, released			14,231,808	100.00
decoder, ours			3,257,344	+22.9
encoder, frozen	encode		27,956,246	-

Table 78. Where every module sits, what shape it is, what it weighs and what share of one released decode it costs; the adapter rows are per adapter and the group rows are for the whole run of groups. Take from it three things. The routable region is groups 2 to 5, which is 60% of the decode and the only part an exit can skip. The adapters and the seam repair add 22.9% to the decoder's parameters and, when one adapter runs, at most 5.31% to its arithmetic. And the deepest exit costs slightly more than the released decoder rather than the same, because it still pays the seam-repair pass.

Source: results/supp_module_shapes.json, derived by scripts/module_shapes.py from the state dict of runs/RECIPE512/ckpt_PAPER.pth.tar (md5 daa3760937ea0bd8) on the CPU. Parameter counts are tensor sizes; the share column is each module's multiply-accumulates per feature pixel over the released decoder's total on that grid, the subpel convolution that opens the decoder being charged at one quarter because it runs on the latent grid. The three aggregate shares that fall out, 8.16% for the upsample, 89.44% for the twelve-block trunk and 2.40% for the head, reproduce the forward-hook audit in scripts/mac_audit.py to four decimals. Section B accounts for the per-exit cost that follows; it is not repeated here.

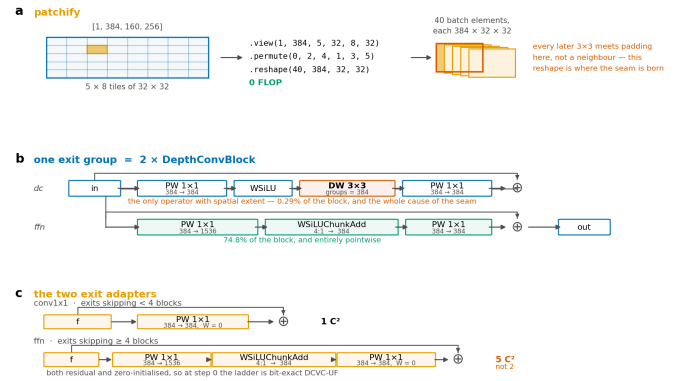


Figure 38. The three pieces a reader has to re-implement. **a**, patchify is a pure reshape and costs nothing, and it is where both the seam and every operation of saving come from. **b**, one exit group is two DepthConvBlocks, whose only operator with spatial extent is a single 3×3 depthwise worth 0.29% of the block, so an early exit gives up almost purely pointwise capacity. **c**, the two adapters that replace it, both residual and both zero-initialised. Shapes are those of a 1080p frame padded to 2048×1280 and split into 40 tiles.

Two of the five adapters can never run: at $j = 2$ the decoder clamps every exit to at least 2, so adapters 0 and 1 are unreachable at inference. They still take gradient from the distillation term, which taps every exit, and they still occupy 1,478,400 of the 3,257,344 parameters we add. Dropping them is the obvious saving in decoder size and is not done here, because the split depth is a configuration knob and a checkpoint that has dropped them cannot be re-run at another j .

A.6. Tiles, and the exit clamp

patchify is a view, a permute and a reshape with no arithmetic at all: a feature map of shape [1, 384, 160, 256] becomes 40 batch elements of [384, 32, 32], as panel a of Figure 38 shows. Tile order is row major, which is what the exit map is indexed by and what flexuf/eval.py reproduces with the same permutation when it measures per-tile error. One tile is 256 px of RGB, 32 px of feature map and 16 px of latent, at 16 pixels per latent position. patchify asserts divisibility rather than padding: an unpadded 1080p frame gives a 40×40 feature map that is not divisible by 32, and the run stops instead of producing a ragged last row.

The clamp is the most consequential line in the decoder. forward() applies exit_map.clamp(min = j, max = K - 1) before the per-tile loop, so a map naming exit 0 at $j = 2$ runs group 2 and wears exit 2's adapter. Four separate places have to apply the same clamp and each was wrong at some point in this project. Getting any of them wrong is silent, which is why they are listed rather than described.

file and function	what it does
decoder.py, forward	clamps the map to [j, K-1] before the per-tile loop
cost.py, exit_costs	charges max(k, j) blocks, and bills the adapter the clamped exit wears rather than the one the map names
eval.py, tiled_exit_mses	fills the columns below j with the decode at exit j, so an argmin over all K lands on a reachable choice
head2.py, forward and oracle_ce_loss	masks the logits below the split depth with -inf, and clamps the oracle label to the same bound

Table 79. The exit clamp, and the four places that have to repeat it. Take from it the failure mode rather than the rule: with the clamp in the decoder but not in the cost model, a router that assigned the shallowest exit everywhere was billed 58.70% saved where the true figure is 42.9%, and that inflated number made the router appear to beat a bound it cannot beat. The mask in the router is -inf and not a large negative constant, because nothing in a cross-entropy penalises a common offset, the raw logits drifted to about -10^4 , and a -10^4 mask then became the largest entry in the row.

Source: the four files named in the first column, under flexuf/. The two percentages are the comment in flexuf/cost.py that records the measurement; no results file carries them, which A.19 repeats.

A.7. The three modules that are ours

The exit adapters. Every exit but the deepest hands the head a corrected feature rather than the raw one, and the two forms Table 78 weighs are built as follows. The plain adapter is $f + \text{conv}(f)$ with one 1×1 whose weight and bias start at zero. The feed-forward adapter repeats the pointwise pair the exit skipped: a 1×1 expanding C to $4C$, the block's own $4:1$ gated chunk-add, and a 1×1 back to C , again zero-initialised. Which form an exit gets is decided by how much it stands in for, the feed-forward form when it skips four or more blocks and the 1×1 otherwise, which at the shipped geometry gives the feed-forward form to exits 0 to 3. Both are entirely pointwise, so neither adds receptive field and neither adds seam, and zero initialisation is what makes the ladder bit-exact against the release at step zero.

The seam repair. One pass over the stitched canvas, after unpatchify and before the head, which is the only point at which a 3×3 can see across a tile boundary. It is a depthwise 3×3 with replicate padding, a gated activation and a zero-initialised 1×1 , and the correction is multiplied by a learned gate indexed by position within a tile: a 32×32 array of scalars shared across all 384 channels, so the module can act on the boundary ring and switch itself off in the interior. The gate is initialised to a decaying function of the distance to the nearest tile edge, so at step zero it already knows where the seams are and training refines rather than discovers. It is charged and not hidden: the cost table bills it at 1.09% of a decode, which is why the deepest exit costs 1.0 rather than exactly one released decode.

The router head. Configuration B predicts the exit map at the decoder instead of reading it from the bitstream. Its inputs are the stem after the shared groups, the decoded latent concatenated with the entropy model's predicted scales, and the quality index; each is projected by a 1×1 and pooled to a mean and a standard deviation per tile, giving $2 \cdot 48 + 2 \cdot 32 + 1 = 161$ numbers that a LayerNorm and three linear layers of width 256 turn into K logits. Only the 1×1 on the stem runs per pixel, while the perceptron runs once per tile, 40 times for a 1080p frame, which is why its width is nearly free. The head is 144 K parameters and 0.163% of the decode. What it learns, what each input is worth and how it compares with a parameter-free rule are section F's subject; only its shape belongs here.

Sources: results/supp_module_shapes.json for the shapes and the seam-repair share, results/router_latency.json for the router's parameter count and its share of the arithmetic. The router checkpoint that file times, runs/RECIPE512/routers2/v2_lam1.3e-5.pth, records runs/RECIPE512/ckpt_eval.pth.tar as the decoder it was trained against, not the pinned one; the six ablation heads of section F are on the pinned checkpoint.

A.8. The deployed decode, and how training differs from it

```

input latent y, quantisation step q, exit map m
1 f = upsample(y)
2 for g = 0 ... j-1: f = groups[g](f) frame, no borders
3 tiles, nh, nw = patchify(f, 32) reshape, 0 MAC
4 m = clamp(m, j, K-1) below j cannot run
5 active = all tiles; work = tiles
6 for g = j ... K-1:
7   work = groups[g](work) borders meet padding
8   leaving = active tiles whose m equals g
9   out = adapters[g](work[leaving]) raw if g = K-1
10  if training: out = out · gate[leaving]
11  canvas[leaving] = out
12  drop leaving from work and from active the saving
13 canvas = unpatchify(canvas, nh, nw)
14 canvas = canvas + G[r mod P, c mod P] · seam(canvas)
15 return pixel_shuffle(head(canvas · q), 8)

```

Table 80. The deployed routed decode, line for line. Take from it that line 12 is the entire saving: every group runs on fewer tiles than the last, and nothing else in the procedure is cheaper than a full decode. Line 10 is the only difference between training and inference, and it is numerically the identity.

Line 10 of Table 80 deserves the space. A router trained jointly with the decoder needs gradient to reach its logits and an argmax blocks it, so the gate is $p / p.\text{detach}()$ with p the selected exit's probability under a Gumbel-softmax relaxation: its value is exactly 1 and the reconstruction bit-identical to the deployed path, while its gradient is the straight-through estimator. At inference the gate is absent and the exit is a hard argmax or a transmitted symbol. That is the one place where the trained object and the shipped object are not the same computation.

A second execution path exists and is bit-identical. With sorted execution the tiles are ordered by depth with a stable argsort before line 6, so "still active at group g " becomes a contiguous prefix, every group is a slice rather than a masked gather, and the loop costs one host read instead of one synchronisation per group. Only the wall clock moves, by 1.2 points, and section B reports it.

A.9. The training data

Training uses Open Images [24], fetched from the CVDF public mirror by scripts/fetch_openimages.sh and prepared by scripts/prepare_openimages.py into the flat layout DCVC-UF's own ImageFolder expects. Of 384,795 files scanned, 380,126 survive a minimum-side filter of 512 px, 4,669 are dropped as too small and 0 as unreadable. The filter is not cosmetic: the loader zero-pads any image smaller than the current crop, and an image that is half black border teaches the decoder to reconstruct black borders. Every 400th entry of the sorted survivor list is then held out, giving 512 validation images and 379,614 training images with no overlap, deterministically, so every run and every control sees the same held-out set.

Each sample is a random crop at the schedule's patch size with a random horizontal flip, converted to YCbCr, scaled to $[0,1]$ and shifted by -0.5 . One quality index is drawn uniformly over the 64 levels per sample and the matching multiplier comes from the log-spaced table running 10 to 2048, so one set of weights covers the whole rate range rather than a point on it. The crop is 512 px, which is what the released recipe's final phase asks for and also the smallest crop carrying more than one tile: at 256 px tiles a 512 px crop holds 4 and a 256 px crop holds one, and a single tile has no border against a neighbour, so patched training at the smaller crop would train a configuration that cannot occur at inference. Evaluation uses a deterministic centre crop with no flip, because the loader's random crop swung one model between 29.54 and 30.58 dB at q0 across two runs of one command.

A.10. The warm start, and the controls it asserts

scripts/warmstart_from_release.py performs the rearrangement, and it is a key remap and nothing else: the opening upsample takes $\text{dec}_1[0]$, group g block i takes $\text{dec}_1[1 + g \cdot b + i]$, the head takes dec_2 , and the adapters and the seam-repair gate are new and zero-initialised. The encoder, hyperprior, entropy model and quantisation-step tables are copied verbatim. The transfer moves 398 tensors and creates 28 adapters at initialisation. Two controls follow from zero initialisation and both are asserted at every build rather than assumed: the deepest exit reproduces the released decoder with a largest absolute difference of 0.0, and every untrained exit equals the released decoder truncated at that depth, also 0.0. A third lives in the checkpoint loader, which tolerates a missing or extra tensor only under the adapter, seam-repair, pad-coefficient and router prefixes and raises otherwise; both sides of a tolerated mismatch are zero-initialised identities, so dropping one is exact rather than approximate.

A.11. The training run

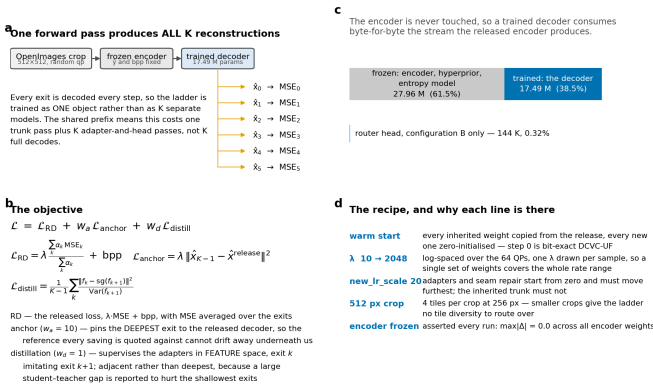


Figure 39. The training scheme. **a**, one forward pass produces all K reconstructions, so the ladder is trained as one object. **b**, the three loss terms. **c**, the encoder is never touched. **d**, why each line of the recipe is there. The figure is a schematic drawn from the model definition; the parameter counts it carries are the ones tabulated in A.5.

The recipe is Microsoft's own, applied at the point in their schedule that matches what we inherit. Their schedule is a 105-entry table of learning rate and crop size indexed by epoch, and the released weights are the output of its last entry, so applying entry zero would kick a converged codec with the from-scratch learning rate. The run enters at offset 99, inside their final 512 px phase at a learning rate of 1×10^{-5} ; all 852 records in the run's log carry that rate and that crop, which is the check that the offset did what it was meant to. Because the offset lands in the phase the experiment needs, no crop-size override is passed at all, where every other warm-started run here raises the crop floor by hand.

One deviation from a plain fine-tune is deliberate. The adapters and the seam-repair gate start at zero and have to move furthest, while the inherited trunk must not move at all if the deepest exit is to stay the released decoder, and the cost of choosing wrongly between them is measured: at the from-scratch rate the trunk drifts 0.20 dB of anchor at q32 in a single epoch. The new modules therefore get their own parameter group at 20 times the schedule's rate, selected by name prefix, with the multiplier re-applied whenever the schedule sets the base rate.

The loss has three terms. The rate-distortion term is the released loss, a multiplier times mean squared error plus bits per pixel, with the error averaged over the exits under fixed weights. The anchor term pins the deepest exit to a frozen copy of the released decoder, because every saving in the paper is quoted against that decoder and a reference drifting underneath the measurement makes the measurement meaningless. The distillation term supervises each adapter in feature space against the exit one step deeper, normalised by the teacher's own variance, following the adjacent-teacher form rather than the deepest-teacher form [17], since a large gap between student and teacher is reported to hurt the shallowest exits most. Gradients are clipped at a norm of 0.1 and a batch whose norm is not finite is dropped; across all 852 records the count of steps skipped for that reason is 0 and the median gradient norm is 0.273.

An epoch is 47,451 optimiser steps, which is 379,614 images at batch 8. The run logs every 200 steps and the median interval is 299.8 s, so a step costs about 1.5 s and an epoch about 19.8 hours. Training is fp32 throughout on a single NVIDIA RTX A6000 with eight data workers.

quality index	epoch 0	epoch 1	change	floor e0	floor e1
q0	29.90	30.84	+0.93	0.0326	0.0295
q16	25.62	27.06	+1.44	0.0429	0.0383
q32	20.93	21.97	+1.04	0.0528	0.0495
q48	18.33	19.92	+1.59	0.0599	0.0551
q63	15.64	17.20	+1.56	0.0659	0.0595
mean	22.08	23.40	+1.31		

Table 81. One more pass over the training set is worth about three points. Saving at the 0.1 dB budget against the released decoder, in percent, on the pinned checkpoint and on the epoch-1 pin, measured with the identical protocol on the identical frames. The last two columns are the floor, the quality already given up before any tile exits early; it falls at every rate. The paper reports the left-hand column, which is therefore a lower bound on what this configuration reaches.

Source: results/signalled_RECIPES12_ctc53.json (runs/RECIPES12/ckpt_PAPER.pth.tar, epoch 0) and results/signalled_RECIPES12_e1.json (runs/RECIPES12/ckpt_PIN_e1.pth.tar, epoch 1), both 53 sequences.

The run had reached epoch 3, step 27,400 of 47,451, when this was written, against the 16 epochs its launcher was given. Table 81 therefore carries a second reading: no comparison in this paper between two runs at different epochs can be read as a comparison between two configurations, and where such a comparison appears it is labelled with both epochs. Nothing in results/ projects the trajectory to convergence and this section does not either; scripts/convergence.py fits a saturating curve with the ceiling pinned but writes only a figure, so its asymptote is not a number this document can cite.

A.12. Every hyperparameter, and where each was measured

knob	value	searched, and where
ladder geometry		
exits K, blocks per exit b	6, 2	signalled_BEST128 and signalled_FINE12, at K = 12
split depth j	2	supp_seam_vs_split, j from 0 to 6
RGB tile	256 px	ctc_seam_p128 against ctc_seam_p256; tilesize_adaptive
halo, latent and trunk	2, 0	no; the trunk halo was priced at 2.25× and rejected
adapter kind	scaled	adapter_ablation
adapter expansion	4	no; the block's own 4:1
seam repair	grid	supp_seam_vs_split
tile padding	replicate	padding_ablation: zeros, replicate, linear, arls
tile coupling	off	coupling_ablation
full-frame head	on	the per-tile arm exists to measure it
decoder training		
optimiser	AdamW	no; the released recipe's
schedule	105 entries, at offset 99	no; verified by scripts/verify_recipe.py
learning rate	1×10^{-5}	no; set by the schedule
multiplier, new modules	×20	no; set from one anchor-drift measurement
crop	512 × 512	no; set by the schedule
batch, workers, precision	8, 8, fp32	no
gradient clip	0.1, non-finite dropped	no; released recipe's
multiplier range	10 to 2048, 64 indices	no; released
quality index	uniform, per sample	no
anchor weight	10	two settings run; 10 kept
distillation	1.0, adjacent	adjacent against deepest
auxiliary weight	1.0, constant	no
training path	deployed tiled decode	against the full-frame arm
encoder	frozen	no; asserted at each evaluation
seed	none set	no; see A.17

knob	value	searched, and where
router head, configuration B		
steps, batch, crop	3,000, 6, 512	no
learning rate, decay	1×10^{-3} cosine, 1×10^{-4}	no
regret weight	1.0	no
seed	0	fixed, so the input ablation is comparable
live inputs	stem, latent, scales, qp	router_ablation, six variants
evaluation		
multiplier bisection	60 halvings on [0, 1]	no
outer correction	6 passes, tolerance 5e-4 dB	no
rate-rank bisection	40 halvings	no
router bisection	on beta, bracket [-2e4, 2e4]	no
timing	40 iterations after 20 of warm-up	no
batch, when timing	1	supp_latency_batch, at 1, 2 and 4

Table 82. Every knob this work sets, its value, and whether it was searched. Take from it that almost nothing on the training side is tuned: the optimiser, the schedule, the clip, the crop and the multiplier range are the released recipe's, and the four settings that are ours exist to protect a property the measurement depends on rather than to improve a number. What was searched is the geometry, and the third column names the file in results/ that holds each comparison, so a reader can price the choice instead of taking it on trust.

Values come from scripts/launch_recipe512.sh, train_flexuf_image.py, scripts/train_router2.py and the four evaluation scripts; the third column names files in results/. The launcher's flags, without the two leading hyphens the shell needs, are: freeze_encoder train_patched epoch_offset 99 new_lr_scale 20 anchor_weight 10 seam_repair grid_adapter_kind scaled_distill_weight 1.0 distill_teacher adjacent lambdas 10 2048 batch_size 8 n 8 e 16 num_exits 6 split_depth 2 latent_patch 16 latent_halo 2 aux_weight 1.0 tile_pad replicate.

A.13. Finding the operating point

A budget is a quality target rather than a multiplier, so the multiplier has to be found, and it is found twice over: an inner bisection on a cheap table of per-tile distortions, and an outer correction against what a real decode delivers. The two levels exist because the table measures each tile with every other tile at the same exit, while a routed frame is mixed and a tile's border sees whatever depth its neighbour chose. The residual between them is small and nearly constant, so shifting the inner target by it converges in two or three passes, and each pass costs one decode per frame rather than one per bisection step.

input budget t in dB, frames F , per-tile cost vector C

```

1 for each frame: cache the [tiles, K] table  $M$  of per-tile error,
   one tiled decode per exit, and the released decoder's frame error  $R$ 
2 floor = trueDb(0) one decode per frame
3 if floor >  $t$ : record unreachable and stop
4 inner =  $t$ 
5 repeat at most 6 times:
6 lo, hi = 0, 1
7 repeat 60 times: on the table, no decode
8 mid = (lo + hi) / 2
9  $k$  = argmin over exits of  $M + \text{mid} \cdot C$ 
10 if tableDb( $k$ ) ≤ inner: lo = mid else hi = mid
11 lam = lo; d = trueDb(lam) one decode per frame
12 if |d - t| < 5e-4: stop
13 inner = inner + (t - d)
14  $k$  = argmin of  $M + \text{lam} \cdot C$ , clamped to  $j$ 
15 saving = 100 (1 - routed MACs / released MACs), by hooks
16 bpp = bpp + mapBits( $k$ ) / pixels

```

Table 83. Finding the operating point, as scripts/signalled_curve.py runs it. Take from it that the arithmetic model is used only inside the argmin on line 9, where a decode per candidate is impossible, and that the number reported on line 15 is counted off the forward pass that actually ran. Line 2 is measured on the deployed path rather than on the table, because the floor is exactly where the tiling penalty is largest.

Three details of Table 83 change what the numbers mean. A rate whose floor exceeds its budget is written out with a reachability flag rather than dropped, so a saturated or unreachable row is visible instead of absent. The saving is measured and not modelled: a forward hook on every convolution and linear layer counts the multiply-accumulates the routed decode performed and divides by the same count for the released decode on the same latent, so the ratio needs no calibration constant and a module skipped by an early exit costs nothing because its hook never fires. The gap between model and meter is reported in every row: at the 0.1 dB budget it runs from 0.43 to 0.69 points, always in the direction of the model flattering us, and it is the measured column the paper quotes.

Two saving columns exist and only one is quoted. One divides by our own deepest exit, the other by the released decoder, and the released decoder is the denominator every claim is stated against; it is exactly 1 in the units of the cost table, so the seam-repair tax stays inside the figure it is supposed to be net of. The same two-level scheme with different constants runs in scripts/raterank_curve.py, scripts/router_curve.py and scripts/hybrid_curve.py, whose constants Table 82 lists.

A.14. How a wall-clock number is taken

An input-adaptive decoder makes wall clock a convention rather than a reading, so the convention is stated before any second is reported. Every timing in this work is at batch 1, on a padded 1920×1080 frame split into 40 tiles, with 20 warm-up iterations discarded and 40 timed iterations interleaved between conditions so that a drift on a shared card lands on all of them equally; the timer is time.perf_counter around a synchronised call. Entropy decoding is outside the timer, because the rate does not depend on the exit and including it would dilute a decoder-side measurement with a constant. The router is charged in arithmetic and not in time, and the two disagree: it is 0.163% of the decode's multiply-accumulates and 0.50% of its median wall clock, a factor of 3.0, because a multiply-accumulate count cannot see a kernel launch.

Batch 1 is the right measurement and not merely a convenient one: a decoder decodes the frame in front of it, and a per-tile-adaptive decoder handed a batch would have to pad every image to the deepest map in that batch, which is a different algorithm. The cost of the choice is measured rather than argued, and so is the trend across a second class of device.

device	batch	ms/frame	realised	predicted	overhead
A6000	1	78.1	29.8%	34.6%	3.5%
A6000	2	76.7	31.0%	34.6%	2.7%
A6000	4	76.5	31.0%	34.6%	3.8%
CPU, 8 threads	1	5625	36.3%	34.6%	-0.9%

Table 84. What the timing convention costs: routed milliseconds per frame at q0 and the 0.1 dB budget, with one exit map throughout, beside the saving realised, the saving the arithmetic model predicts and the cost of the tiled path itself. Take from it two things. Batch 1 gives up about one point of realised saving against batch 4, so the convention is cheap and stating it is cheaper. And the gap between arithmetic and the clock is a scheduling effect rather than an arithmetic one: on the GPU the same map realises less than the multiply-accumulate model predicts, while on a CPU it realises more and the tiled path costs nothing at all. The device this paper reports is the one that flatters the method least.

Sources: results/supp_latency_batch_1920x1080.json and results/supp_latency_cpu_1920x1080.json, both on the pinned checkpoint with the exit map read from results/supp_paper_curve_PAPER.json. The GPU rows were taken under the evaluation lock on a shared card carrying four other people's training runs; the CPU rows used 8 threads on the same shared machine and are not an idle-box measurement. All eight cards on this machine are the same model, so a CPU is the only second device class reachable here.

A.15. What the work cost to produce

item	cost	source
one decoder epoch	19.8 h on one A6000	train_log.jsonl
the pinned checkpoint	1 epoch	ckpt_PAPER records epoch 0
the run so far	3 epochs and 27,400 steps	train_log.jsonl
one router head	3,000 steps, 1.03 h	router_ablation_stem
the router input ablation	6 heads, about 6.2 h	the six router_ablation files
encoder search, exact	4.52 decodes per frame	supp_encoder_cost
encoder search, approximate	1.37 decodes per frame	supp_encoder_cost

Table 85. What producing these numbers costs, priced rather than asserted. Take from it that the encoder pays for the allocation and the decoder does not: the exact per-tile table costs 4.52 decodes per frame against 1.37 for the full-frame approximation, which agrees with the exact search on 82.5% of tiles and gives up 0.68 points of saving. Nothing on the decoder side is searched at all, which is the point of signalling the map.

Sources: results/router_ablation_stem.json and its five siblings for the router timings, results/supp_encoder_cost_PAPER.json for the search cost (pinned checkpoint, q63, 40 tiles, 15 iterations, one tiled decode at 114.9 ms). Decoder wall clock is from runs/RECIPE512/train_log.jsonl; no file in results/ records it.

A.16. The pinned checkpoint, and what every table rests on

Results are measured on runs/RECIPE512/ckpt_PAPER.pth.tar, md5 daa3760937ea0bd89b89c555c2eefa88, which records epoch 0 inside itself and is byte identical to that run's ckpt_epo0.pth.tar. The name exists because a filename is not a checkpoint here: a per-epoch watcher overwrites runs/*/ckpt_eval.pth.tar as each epoch lands, so a results file naming that path names a file that no longer exists. scripts/make_paper_tables.py and scripts/check_paper.py both hold the pinned name and, among candidates for one quantity, prefer a file whose own ckpt field records it, falling back to the most recently written; hand-written order breaks ties and nothing else. That is a fix rather than a convenience, because an earlier hand-ordered list kept preferring a superseded file after the pinned one had been remeasured.

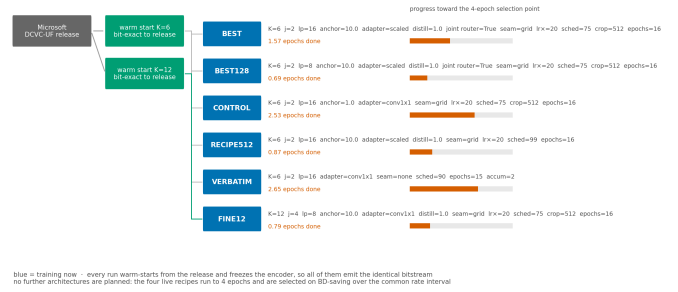


Figure 40. Lineage of the training runs. Every run warm-starts from the released decoder through one of two remaps, one per ladder size, and freezes the encoder, so all of them consume the identical bitstream. The flags shown are the ones that differ between runs. The progress bars are a snapshot taken on 2026-08-18 and the runs have advanced since; the current position of the run this paper reports is given in A.11.

results file	backs	checkpoint	n
signalled_RECIPE512_ctc53	main results, complexity	pinned, e0	53 seq × 1
saturation_RECIPE512_ctc53	operating range	pinned, e0	53 frames
static_RECIPE512_b01	static controls	pinned, e0	53 seq
per_class_RECIPE512	per class	pinned, e0	53 frames
router_RECIPE512_b01	A against B	pinned, e0	53 seq × 1
router_RECIPE512_b03	A against B	pinned, e0	53 seq × 1
raterank_RECIPE512_b01	rate rank	pinned	53 seq × 1
raterank_RECIPE512_b03	rate rank	pinned	53 seq × 1
raterank_RECIPE512_b05	rate rank	RECIPE512/eval	53 seq × 1
hybrid_RECIPE512_b01_fixed	partial signalling	pinned	53 seq × 1
hybrid_raterank_b01	rate rank, second run	pinned	53 seq × 1
hybrid_v3_b01	rate rank, second run	RECIPE512/eval	53 seq × 1
latency_RECIPE512_sorted	latency	RECIPE512/eval	-
router_latency	operating range	RECIPE512/eval	-
adapter_ablation	adapter ablation	RECIPE512/eval	16 frames
combined_RECIPE512_b01	blend	RECIPE512/eval	53 seq × 1
hybrid_RECIPE512_b03_fixed	macros only	RECIPE512/eval	53 seq × 1
hybrid_lorenz_b01	macros only	RECIPE512/eval	53 seq × 1
coupling_ablation	macros only	RECIPE512/eval	16 frames
map_transfer	macros only	RECIPE512/eval	-
band_collapse	macros only	not recorded	-
band_collapse_BEST	macros only	not recorded	-
router_retrain_compare	macros only	not recorded	-
signalled_BEST_ctc53	ladder configurations	BEST/eval, e1	53 seq × 1
signalled_BEST128_ctc53	ladder configurations	BEST128/step, e1	53 seq × 1
signalled_FINE12_ctc53	ladder configurations	FINE12/step, e1	53 seq × 1
ctc_seam_p256	padding, tile size	not recorded	-

results file	backs	checkpoint	n
ctc_seam_p128	tile size	<i>not recorded</i>	-
raterank_BEST_compare	rate rank, second run	<i>not recorded</i>	-
bdrate	BD-rate	<i>not recorded</i>	-
check_paper	claim check	<i>not recorded</i>	-

Table 86. Provenance of every generated table in the main paper. These are the 31 results files scripts/make_paper_tables.py reads, and the checkpoint column is read from each file at build time rather than typed. Take from it that 10 of the 31 are on the pinned checkpoint and the rest are not, so a reader comparing two rows of two different tables should check this column first. "eval" is the overwritten per-epoch file, and "not recorded" means the file carries no checkpoint field at all, which is a defect in the script that wrote it.

Three groups in Table 86 deserve separate comment. The pinned group carries the headline saving, the operating range, the per-class breakdown, the static controls and both routed configurations, so the paper's central claim rests on one epoch of one run measured consistently. The eval group is measured on a file that has since been overwritten, so those numbers can be quoted but not reproduced; the latency table and the adapter ablation are in it. The group with no checkpoint field at all cannot even say which decoder it describes.

A.17. Seeds, variation, and the interval that does exist

No seed is set anywhere in the decoder's trainer. Four things vary between two runs of one command: the sampler is reshuffled every epoch from the global generator, the quality index is drawn per sample, the tiled training path draws a fresh random exit for every tile at every step, and cuDNN selects kernels by autotuning. The honest statement is that the recipe reproduces and the run does not, and this work has no repeated run of one configuration from which a training-side spread could be quoted. The router experiments are the exception and were built to be: every variant of the input ablation fixes seed 0 and shares architecture, parameter count, optimiser, image order and quality-index draws, so the only difference between them is which inputs the head may see, and their agreement figures carry a standard error across 1,200 frames.

quality index	mean	sd	min	median	max	worst sequence
q0	32.70	7.51	6.04	35.89	39.12	BQSquare
q16	28.81	10.60	-0.95	31.67	39.12	BQSquare
q32	24.58	12.04	-0.95	26.10	39.12	PartyScene
q48	22.40	11.75	-0.95	23.54	39.12	BlowingBubbles
q63	20.00	11.13	-0.95	20.39	39.12	RaceHorses

Table 87. The interval this work can honestly report: saving in percent against the released decoder at the 0.1 dB budget, across the 53 test sequences, on the pinned checkpoint. Take from it that the spread is content and not noise. It widens with rate, from a standard deviation of 7.5 points at q0 to 11.1 at q63, and the minimum turns negative at the high rates because a sequence whose tiles all stay at the deepest exit costs about 1% more than the released decoder rather than less. This measures the test set, not the training draw, and is not a substitute for the run-to-run interval that does not exist.

Source: results/supp_per_sequence_PAPER_b010.json, the rows_vs_release block, computed from results/supp_paper_curve_PAPER.json on runs/RECIPE512/ckpt_PAPER.pth.tar. The file carries the same spread in our own deepest-exit denominator as well; the two differ by more than rounding and are never mixed.

quality index	path 1	path 2	gap
q0	35.50	33.10	2.40
q16	30.32	27.58	2.74
q32	22.15	19.82	2.33
q48	16.88	15.14	1.74
q63	7.65	8.67	-1.01

Table 88. Two implementations of the same quantity do not agree. Saving in percent at the 0.1 dB budget, computed by the curve-sweeping path and by the signalling path, which differ in how they pool distortion across tiles and frames. Take from it a floor on protocol sensitivity: the gap reaches 2.74 points, larger than several of the differences the paper draws conclusions from, so numbers from the two paths are never mixed inside one table.

Source: results/crosscheck_paths.json. The file records no checkpoint field.

Evaluation itself is deterministic once a checkpoint is fixed: the sequence list, the leading frames and the tile grid are all fixed and no crop is random. scripts/check_paper.py re-reads 85 numerical claims out of the paper's prose and checks each against the file it is supposed to come from. As recorded in results/check_paper.json, 84 of them pass and one does not: "hybrid: best margin over A (pts)". At its endpoint the partially signalled configuration reduces to the fully signalled one by construction, and on the re-measured data it does not quite: it finds a genuinely cheaper allocation at the same delivered quality, by 0.04 to 0.21 points, which bisection noise is too small to explain. It is open at the time of writing and is listed among the limitations rather than presented as passing.

A.18. Data, code and provenance

- **Training images.** Open Images [24], from the CVDF public mirror named in scripts/fetch_openimages.sh, prepared into /data10/shareddata/openimages/dvc_train. The terms are the ones that mirror publishes and nothing here redistributes them.
- **Test sequences.** UVG [25] from ultravideo.fi, MCL-JCV [26] and HEVC classes B, C and D from the public mirrors named in scripts/fetch_mcljcv.sh and scripts/fetch_hevc_bcd.sh, and HEVC class E from media.xiph.org. Each fetch script records its source URL, and no sequence is re-encoded before it is measured.
- **The released codec.** The DCVC-UF intra checkpoint [14], identified by sha256 in results/dmc_ld_recon_audit.json and used under the licence its repository ships.
- **Our code and checkpoints.** The trainer, the ladder, the cost model, the meter, the router and every evaluation script named here are in the repository this supplement is built from; the pinned checkpoint is identified by md5 in A.16.
- **Human subjects.** None, which is why A.4 declines to claim that 0.1 dB is invisible.

A.19. What no file in results/ pins

The rule this supplement follows is that a number is quoted only with the file it came from. Applying it honestly means listing the places where the file does not exist.

- **The decoder's training recipe.** Nothing that writes to results/ runs inside the trainer, so no results file records the learning rate, the schedule, the batch size, the crop, the loss weights or the number of steps. Table 82 is read from scripts/launch_recipe512.sh, train_flexuf_image.py and the run's own log, and the dataset statistics of A.9 from a prepare_stats.json that lives outside the repository.
- **The wall clock.** Read from runs/RECIPE512/train_log.jsonl, which a live process appends to, so the position quoted in A.11 and the hours in Table 85 are a snapshot taken at 2026-08-20 12:48.
- **The warm start's own controls.** runs/warmstart/warmstart_report.json holds the transfer counts and both zero-difference controls, but the copy on disk is the K = 12 rebuild of 17 August and the K = 6 report the pinned run descends

from was overwritten. The controls are asserted inside the script at every build, so a failure would stop the run, but the $K = 6$ record is gone.

- **The exit-clamp mis-billing.** The two percentages in Table 79 are a comment in flexuf/cost.py recording a measurement made once and never written out.
- **A repeated training run.** With no seed and no configuration trained twice, no run-to-run interval exists. Table 87 is the nearest substitute and measures the test set, and Table 88 measures the evaluation path; neither measures the training draw.
- **A second class of accelerator.** All eight cards here are the same model, so the second device in Table 84 is a CPU.
- **Any resolution above 1080p.** The queued 4K latency job returned without writing a file and results/supp_footprint.json marks its 3840×2304 row oom, so nothing here is measured at 4K.
- **A second decoder.** The ladder needs only a residual trunk with a shared head, and no second codec was rearranged into one, so the generality of the construction is argued and not measured.

B. Architecture and complexity accounting

This section states what the decoder is, module by module and convolution by convolution, derives what a decode costs from those shapes, and then checks the derivation against counts taken off decodes that were run. It closes with what the saving is worth in the units a deployment measures, which are milliseconds, joules, megabytes and frames per second, and none of which is the unit the allocation is optimised in.

Some vocabulary first, because the rest of the supplement uses it. A *multiply-accumulate* (MAC) is one multiply and one add, and MAC/px is per pixel of the grid the operator runs on, which is not the same grid for every operator. The *trunk* is the released decoder's stack of $N = 12$ identical DepthConvBlocks at width $C = 384$. An *exit* is a depth at which a tile may stop; the ladder has $K = 6$ of them, one every $b = N/K = 2$ blocks. The *split depth* $j = 2$ is how many exits' worth of blocks run once for the whole frame before any tile is allowed to leave, so the first $jb = 4$ blocks are full-frame and the last 8 are per-tile. An *adapter* is a small zero-initialised residual module that stands in for the blocks an exit skips. A *tile* is 256 RGB px square, which is 32 px of the feature grid and 16 px of the latent. The *exit map* is the per-tile assignment of exits. The *released decoder* is DCVC-UF's intra decoder [14] with none of this added, and it is the unit every cost below is quoted in.

B.1. The decoder, module by module

The released intra decoder is 14,231,808 parameters in 72 convolutions and nothing else: no attention, no normalisation with parameters of its own on the critical path, and no operator that changes the spatial grid except two pixel shuffles. It has three parts. An *upsample* takes the decoded latent, widens it with a 1×1 and doubles its grid with a pixel shuffle, then runs one DepthConvBlock at $C = 384$. A *trunk* of 12 further DepthConvBlocks runs at that width and grid throughout. A *head* narrows to 192 channels with a 1×1 , runs one more DepthConvBlock at that narrower width, and pixel-shuffles by 8 straight to RGB, with no convolution after the shuffle. Table 89 is that pipeline with its shapes.

Module	Where	Grid	Out ch.	MAC/px	% dec.
Latent y, decoder input	entropy dec.	68×120	256	0	0.00
1×1, widen	upsample	68×120	1536	393,216	0.71
PixelShuffle 2	upsample	136×240	384	0	0.00
DepthConvBlock	upsample	136×240	384	1,035,648	7.45
DepthConvBlock × 12	trunk	136×240	384	1,035,648	89.44
1×1, narrow	head	136×240	192	73,728	0.53
DepthConvBlock	head	136×240	192	259,776	1.87
PixelShuffle 8	head	1088×1920	3	0	0.00
Ours, added					
Conv1x1Adapter	exit 4	32×32	384	147,456	1.06
FFNAdapter	exits 0 to 3	32×32	384	737,280	5.31
GridSeamRepair	post-stitch	136×240	384	150,912	1.09
Router head	stem	per tile	$K = 6$	n/a	0.163

Table 89. The decode, module by module, at a 1920×1080 frame padded to 1920×1088. Grids are height × width, and “Where” names the released decoder's three parts: the upsample dec\1[0], the trunk dec\1[1..12] and the head dec\2. The last column is the module's share of one released decode, and for the two adapters it is the share if every tile wears that adapter. Take from the table that the grid changes exactly twice and never inside the trunk, so an exit ladder cut into the trunk needs no resampling; that the head is cheap because it runs at 192 channels rather than 384, which is a quarter of the arithmetic per pixel; and that the FFN adapter is the most expensive thing this work adds, at 5.31% of a decode, more than half a trunk block.

Released modules and their shapes: results/mac_audit.json, forward hooks on every convolution of an IntraDecoder at the real tensor shapes. A MAC count depends on shapes and not on values, so no weights are loaded for it. Adapters and seam repair: results/adapter_cost.json, hooks on the modules of runs/RECIPE512/ckpt_PAPER.pth.tar. Router: results/router_latency.json, measured on runs/RECIPE512/ckpt_eval.pth.tar; the head is the same size in both checkpoints.

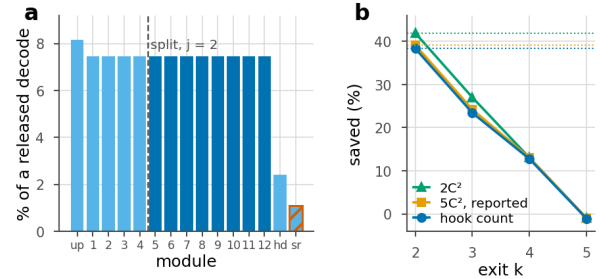


Figure 41. Where the arithmetic is, and what an exit saves. **a**, one released decode by module: the opening upsample, the twelve trunk blocks, the head, and the seam-repair pass this work adds (hatched). Light bars always run; dark bars are the eight blocks after the split that a tile can skip. **b**, the saving at each reachable exit under three prices for the same architecture, with each price's ceiling dotted. Section B.5 is the table behind it.

B.2. Every convolution in the decode

Table 90 gives every convolution in the decode in the (K, C_{in}, C_{out}, S) convention that the architecture figures of this literature use, with the group count added because two of the shapes are depthwise. The 72 convolutions of the released decoder have only 8 distinct shapes between them, because the DepthConvBlock repeats.

Convolution	K	C _{in}	C _{out}	S	Grp	MAC/px	Used
up.conv.0	1	256	1536	1	1	393,216	1
dc.0, 1×1 in	1	384	384	1	1	147,456	×13
dc.2, 3×3 depthwise	3	384	384	1	384	3,456	×13
dc.3, 1×1 out	1	384	384	1	1	147,456	×13
ffn.0, 1×1 C→4C	1	384	1536	1	1	589,824	×13
ffn.2, 1×1 4C→C	1	384	384	1	1	147,456	×13
dec_2.adaptor	1	384	192	1	1	73,728	1
head dc.0	1	192	192	1	1	36,864	1
head dc.2, depthwise	3	192	192	1	192	1,728	1
head dc.3	1	192	192	1	1	36,864	1
head ffn.0	1	192	768	1	1	147,456	1
head ffn.2	1	192	192	1	1	36,864	1
Conv1x1Adapter	1	384	384	1	1	147,456	1
FFNAdapter pw_in	1	384	1536	1	1	589,824	×4
FFNAdapter pw_out	1	384	384	1	1	147,456	×4
Seam repair, 3×3 dw	3	384	384	1	384	3,456	1
Seam repair, 1×1	1	384	384	1	1	147,456	1

Table 90. Every distinct convolution in the decode. K is the kernel side, S the stride, Grp the group count; MAC/px is per pixel of that operator’s own grid, which is the latent grid for the first row and the feature grid for the rest. “Used” counts the instances in one decode: the DepthConvBlock appears 13 times, once in the upsample and 12 times in the trunk, and there are $K - 1 = 5$ adapters of which four are of the FFN kind. Take from it that only two of the shapes have a kernel wider than 1×1 and both are depthwise, so the 14 instances of them in one decode carry 0.34% of the arithmetic between them.

results/mac_audit.json for the released rows, results/adaptor_cost.json for ours. The two depthwise convolutions are the whole of the decoder’s spatial footprint, which is why a tile boundary is cheap to repair.

That last figure is the premise of the whole method, so it is worth stating separately: 99.66% of the released decoder’s arithmetic is 1×1 convolution, which has no neighbours to miss. The receptive field of the trunk plus head grows by exactly one feature pixel per block, from 1 feature pixel with no trunk blocks to 13 with all 12, which is 104 RGB px per tile edge (results/dmc_ld_rf_probe.json, measured on the released DCVC checkpoints rather than ours). A tile decoded alone is therefore wrong only in a band, and the band is narrower for a tile that exits early.

The two DepthConvBlock widths obey the same closed form. Writing the block as a 1×1 projection, a 3×3 depthwise, a second 1×1 and a feed-forward pair whose first 1×1 expands to 4C and whose gated activation folds 4C back to C for free, the closing 1×1 is C→C rather than 4C→C and the block is

$$M_{\text{block}}(C) = C^2 + 9C + C^2 + 4C^2 + C^2 = 7C^2 + 9C \quad (37)$$

which is 1,035,648 MAC/px at $C = 384$ and 259,776 at $C = 192$, both matching Table 90 row by row. The cost model shipped in flexuf/cost.py writes it as $8C^2 + 9C = 1,183,104$ instead, one C^2 too many, which overstates a block by 14.24%. Sections B.5 and B.6 follow that one term to the end.

B.3. What the ladder adds

Three modules are new. An exit that stops early hands the head a feature map the remaining blocks would have refined, and an *adaptor* stands in for them. The 1×1 adaptor is $f + Wf$ with W a $C \times C$ matrix. The FFN adaptor repeats the block’s feed-forward pair, a 1×1 expanding $C \rightarrow 4C$, the same gated activation, and a 1×1 $C \rightarrow C$. Both are zero-initialised, so the ladder is the released decoder exactly at step zero and any quality the adaptors add is attributable to them. Neither has a spatial footprint, so neither widens the seam. The *seam repair* is a gated depthwise 3×3 followed by a zero-initialised 1×1, run once on the stitched canvas after unpatchify and before the head, which is the only place a 3×3 can see across a tile boundary. The *router head* reads the shared stem and predicts a per-tile exit; Section F is its architecture and this section only prices it.

Module	Operators	MAC/px	In C ²	Blocks	Of decode
1×1 adaptor	1×1 C→C	147,456	1.00	0.1424	1.061%
FFN adaptor	1×1 C→4C, 1×1 C→C	737,280	5.00	0.7119	5.306%
Seam repair	3×3 dw, 1×1 C→C	150,912	1.02	0.1457	1.086%
Router head	two 1×1, then an MLP	n/a	n/a	n/a	0.163%

Table 91. What the ladder adds, priced against one released decode. “Blocks” is the module in units of a trunk block. Take from it that the FFN adaptor costs $5C^2$ and not $2C^2$: the gated activation is free, so the pair is $4C^2 + C^2$ rather than $4C^2 + 4C^2$, and the module is 0.71 of a whole block. Since the routed allocation lives at exits 2 and 3, which both wear the FFN adaptor, that correction lands on precisely the exits the reported saving leans on.

results/adaptor_cost.json for the two adaptors, on the pinned checkpoint. The seam-repair row is $9C + C^2$ per pixel; Section B.6 confirms that figure against a hook count rather than taking it from the source. Router: results/router_latency.json.

The ladder assigns the FFN adaptor to any exit skipping four or more blocks and the 1×1 adaptor otherwise, which at $K = 6$ and $b = 2$ means the FFN at exits 0 to 3, the 1×1 at exit 4, and no adaptor at exit 5, whose feature is the trunk’s own output with nothing standing in for a skipped block. The arithmetic at exit 5 is therefore the released decoder’s, plus the repair pass, which is why the deepest exit costs slightly more than the release rather than the same.

B.4. From the block to the decoder

Write s_{up} , s_{trunk} and s_{head} for the shares of a released decode taken by the upsample, the trunk and the head; a_k for the adaptor at exit k ; and r for the seam-repair pass. A tile assigned an exit shallower than j still runs group j , because the decoder clamps the map, so the cost of exit k in units of one released decode is

$$C_k = s_{\text{up}} + s_{\text{head}} + r + s_{\text{trunk}} \frac{(\max(k, j) + 1)b}{N} + a_{\max(k, j)} \quad (38)$$

and the frame-level cost of an exit map a over T tiles amortises the shared part over the frame rather than over the tile,

$$C(a) = s_{\text{up}} + s_{\text{trunk}} \frac{jb}{N} + s_{\text{head}} + r + \frac{1}{T} \sum_{i=1}^T u_{a(i)} \quad (39)$$

with u_k the per-tile suffix, meaning the blocks after the split plus the adaptor. Only the last term depends on the map. That is the structural reason a ladder over a decoder of this shape has a ceiling well below 100%: at the split depth the upsample, the first $jb = 4$ blocks, the head and the repair pass are already 41.5% of a released decode, and they run whatever the map says.

None of the four shares has to be taken on trust. results/mac_audit.json gives $s_{\text{up}} = 0.0816$, $s_{\text{trunk}} = 0.8944$ and $s_{\text{head}} = 0.0240$ by hook count, so one trunk block is 0.074533 of a decode. That figure has an independent check. results/static_RECIPE512_b01.json records the frame-level saving of a map that sends every tile to exit e , for $e = 2$ to 5, on the pinned checkpoint; differencing consecutive entries gives one trunk block as 0.074533, which agrees with the audit to 7×10^{-7} of a decode. One is a shape audit of the released decoder with no weights loaded and the other is a set of measured savings on ours, which is the reason to report both.

B.5. The per-exit cost vector and the ceiling

Exit k	Blocks	Adapter	c (2C ²)	c (5C ²)	c hook	Saved %
0 (clamped)	6	FFN	0.5809	0.6088	0.6167	38.33
1 (clamped)	6	FFN	0.5809	0.6088	0.6167	38.33
2	6	FFN	0.5809	0.6088	0.6167	38.33
3	8	FFN	0.7300	0.7578	0.7658	23.42
4	10	1×1	0.8697	0.8697	0.8724	12.76
5	12	none	1.0095	1.0095	1.0109	-1.09

Table 92. The per-exit cost vector, in units of one released decode, under three prices for the same architecture: the first shipped table, which put the FFN adapter at 2C²; the table the paper reports, which corrected that to 5C² but kept the closed-form block; and the hook count assembled in Section B.4. Take from it that the two corrections move the shallowest reachable exit by 2.79 and then 0.80 points and leave exits 4 and 5 nearly untouched, because those wear the cheap adapter or none, and that the deepest exit costs 1.09% more than the release, which is the seam-repair pass.

Exits 0 and 1 are unreachable: the decoder clamps the map at $j = 2$, so a tile nominally assigned them leaves through exit 2 and wears exit 2’s cost. The 2C² and 5C² vectors are recorded in results/why_qp.json and results/why_qp_PAPER.json; a cost vector is a property of the model rather than of the weights, so the first being measured on runs/BEST/ckpt_eval.pth.tar does not affect it.

The ceiling is $100(1 - c_5)$, the saving when every tile takes the shallowest reachable exit. It is a property of the architecture and no allocation can pass it. Three values are in circulation and they are the three columns of Table 92: 41.91% at 2C², 39.1% at 5C², which is the value the paper’s tables use, and 38.33% by hook count. The last of the three is the one a meter agrees with, and the next subsection is the demonstration.

B.6. Predicted against hook-measured

A cost model and the decoder it models drift apart. The check that catches it is a hook count: flexuf/measure.py attaches a forward hook to every convolution and linear layer and accumulates MACs from the output shape of each call. A group skipped by an early exit never fires its hook and so costs nothing; a group run on 28 of 40 tiles costs 28 tiles’ worth, because tiles are the batch dimension and the output shape carries them. There is no bookkeeping to get wrong, and the count is the ground truth the model is judged against.

Condition	Map 2/3/4/5	Measured	5C ²	diff	Hook	diff
720p q0	11/4/0/0	34.3524	35.1494	+0.797	34.3524	+0.0000
720p q32	6/4/2/3	23.0606	23.6547	+0.594	23.0606	+0.0000
720p q63	4/3/4/4	18.0177	18.4971	+0.479	18.0177	+0.0000
1080p q0	28/10/1/1	32.9763	33.7436	+0.767	32.9763	+0.0000
1080p q32	16/10/6/8	22.8829	23.4682	+0.585	22.8829	+0.0000
1080p q63	11/7/11/11	17.8489	18.3184	+0.470	17.8489	+0.0000

Table 93. Predicted against measured, on six decodes with six different exit maps at two resolutions. “Measured” is the hook count off the decode that was executed and timed. Take from it that the assembly of Section B.4 reproduces the count exactly in every condition, to four decimal places and in fact to floating point, while the model the paper reports is optimistic by 0.47 to 0.80 points and always in the same direction.

results/supp_power.json, pinned checkpoint, 0.1 dB budget, NVIDIA RTX A6000. The map is the file’s pooled exit histogram rounded to the frame’s tile count, which is what scripts/power_profile.py decodes; its four columns are the tiles leaving at exits 2, 3, 4 and 5.

Because a hook count is exactly linear in the exit map, six mixtures spanning four reachable exits over-determine the per-exit vector and it can simply be solved for. The six equations are consistent to 3×10^{-16} , and the solution differs from the assembled vector by at most 2×10^{-15} at any exit. The identity is worth stating plainly: the per-module counts of results/mac_audit.json and results/adapter_cost.json are not a model of what the meter reports, they are what the meter reports, rearranged. Where the paper’s tables differ from the meter, the difference has a closed form and Table 94 gives it term by term.

Term	Model price	Hook price	Points of saving
Seam repair	0.009507	0.010861	0.1354
1×1 adapter	0.009289	0.010612	0.1323
FFN adapter	0.046447	0.053060	0.6613
Exit 5 residual			0.1354
Exit 4 residual			0.2678
Exits 2, 3 residual			0.7970

Table 94. The residual of Table 93, term by term. Three modules are priced as a fraction of a block, and the shipped model’s block is $8C^2 + 9C$ where the meter’s is $7C^2 + 9C$. Take from it that the residual is not noise and not a tolerance: exit 5 wears the repair mispricing alone, exit 4 wears the repair plus the 1×1 adapter, and exits 2 and 3 wear the repair plus the FFN adapter, which is why the error is largest exactly where the saving is largest.

Model prices: the module’s MAC/px divided by $8C^2 + 9C$, times one trunk block. Hook prices: the module’s MAC/px times the feature grid, divided by the decode total, both from results/mac_audit.json and results/adapter_cost.json. The three residual rows are the corresponding columns of Table 92.

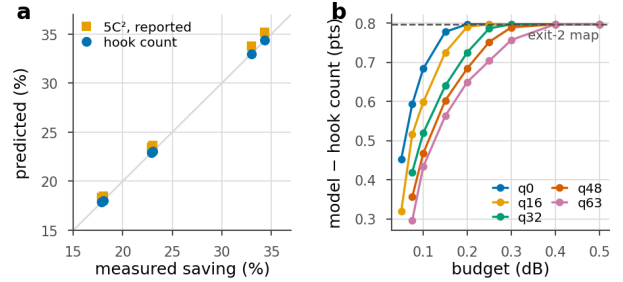


Figure 42. The model against the meter. a, six decodes at two resolutions: the hook-count assembly lies on the identity line and the model the paper reports lies above it. b, the same residual over the whole budget grid, 9 budgets by 5 quality indices on 53 sequences. It is positive everywhere, grows with the budget, and saturates at the value an all-exit-2 map implies.

Figure 42b runs the same comparison over the whole operating range rather than six points of it. The residual is positive in all 42 cells, from 0.296 to 0.797 points, and it saturates at exactly the exits-2 value of Table 94. Reading the paper’s headline figures as hook counts therefore means subtracting between 0.43 and 0.69 points: at the 0.1 dB budget the model reports 29.9% at the lowest rate against a measured 29.21%, and 15.6% at the highest against a measured 15.20%.

results/signalled_RECIPE512_grid.json and results/signalled_RECIPE512_ctc53.json, both on the pinned checkpoint. Both columns are in the files: saving_pct_vs_release is the model, saving_pct_measured is the hook count, and model_minus_measured is their difference.

B.7. How the timings were taken

The rest of this section leaves arithmetic for the clock, so the protocol comes first. Everything below follows the same rules and they are stated rather than implied.

- **Batch 1, and why.** Every wall clock here is one frame at a time, which is what a decoder does. A batch is genuinely different work for this decoder and not a repetition, because exit_map is indexed over the whole batch’s tiles, so B frames of 40 tiles form one shrinking active set of 40B and the groups run wider. Section B.8 measures what the choice costs.
- **Warm-up, then interleaving, then the median.** 20 untimed iterations, then 40 rounds of one call of each variant in turn. This is a shared card, and timing all of the released decoder and then all of ours puts any drift in a neighbouring job’s load straight into the ratio, which is the only quantity these runs exist to produce.
- **What the timer wraps.** time.perf_counter around a synchronised call, not CUDA events, because events do not exist on the CPU and the two device classes have to be timed alike. The timer starts at the decoded latent and stops at the reconstruction, so it includes patchify, every group, unpatchify, the seam-repair pass and the head.

- **What is deliberately outside it.** Entropy decoding, because the bitstream and therefore the rate do not depend on the exit map: including it would divide both sides of every ratio by the same constant and flatter the method. The map itself is charged, in bits rather than in time, at 89 bits per frame.
- **The router is charged in MACs and not in seconds.** Its arithmetic is 0.163% of a decode but 0.50% of decode time, $3.0\times$ its arithmetic share, an extra 0.33 points bought by launching a small kernel over a large map. That is the general reason a MAC count cannot see a kernel launch, in miniature. The router does not run at all in the signalled configuration, where the encoder chooses the map.
- **The masked path, not the sorted one.** The shipped configuration leaves sorted_tiles off, so every group is a masked gather over the full tile batch. Sorting the tiles by depth turns each group into a contiguous slice and recovers 1.2 points at 1080p for bit-identical output (results/latency_RECIPE512_sorted.json, on runs/RECIPE512/ckpt_eval.pth.tar). None of the timings below use it, so they are the slower of the two implementations.

B.8. Where the seconds go, and on what device

Stage	MAC %	1080p tiles	1080p ms	720p tiles	720p ms
Stem: upsample + groups 0, 1	37.97	frame	41.39	frame	16.30
Patchify	0.00		0.21		0.09
Group 2 (2 blocks, per tile)	14.91	40	15.98	15	6.19
Group 3 (2 blocks, per tile)	14.91	28	11.24	10	4.16
Group 4 (2 blocks, per tile)	14.91	14	5.72	5	2.23
Group 5 (2 blocks, per tile)	14.91	9	3.79	3	1.30
Unpatchify	0.00		0.21		0.09
Seam repair (full-frame)	1.09	frame	2.07	frame	0.83
Head	2.40	frame	4.36	frame	1.72

Table 95. Where a routed decode goes, in arithmetic and in milliseconds. The MAC column is each stage's share of a released decode at full occupancy and sums to 101.09%, which is the deepest exit's cost. Take from the table that patchify and unpatchify are free arithmetically and nearly free on the clock, that the repair pass is 1.09% of the arithmetic and 2.4% of this decode's clock, and that the head takes 5.1% of the clock for 2.40% of the arithmetic, being bandwidth-bound rather than arithmetic-bound.

results/supp_latency_1920x1080.json and results/supp_latency_1280x720.json, pinned checkpoint, q32, 0.1 dB budget, NVIDIA RTX A6000 taken under the evaluation lock so the card was not shared. Tile counts are that decode's own exit map, which leaves 28, 14 and 9 of 40 tiles alive in groups 3, 4 and 5.

Patchify and unpatchify are reshapes and cost no arithmetic at all. They require the feature map to divide by the 32 px feature tile, which the encoder-side replicate padding guarantees: 1920×1080 becomes 2048×1280 and 40 tiles, 1280×720 becomes 1280×768 and 15 (results/tile_definition.json).

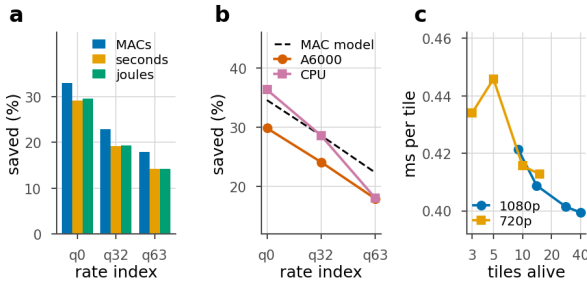


Figure 43. What the arithmetic is worth. a, one exit map measured three ways at 1080p. b, the same map on two device classes against what the MAC model predicts: the A6000 falls short of the prediction at every rate, the CPU does not. c, why a GPU falls short: a tile costs more to decode in a group that has fewer tiles left in it.

Two effects separate seconds from multiply-accumulates and the figure isolates both. The first is the tiled path itself, which the released decoder does not pay: patchify, unpatchify and the repair pass are 2.9% of the routed clock at 1080p. Timing our own decoder with every tile at the deepest exit isolates it: that variant does the same arithmetic as the release plus the tiling machinery, and it is 2.6 to 3.8% slower across nine measurements on the pinned checkpoint, against 3.6% for the figure the main paper quotes. The second is occupancy. A group late in the ladder launches kernels over a handful of 32×32 feature maps, and a GPU sized for the first group is partly idle inside them: at 1080p a tile costs 5.5% more in group 5, on 9 tiles, than in group 2 on 40. At 720p the same curve rises by 8.0% from 15 tiles to 5 and then falls back at 3, so the trend is clear at 1080p and not monotone at 720p.

Device class and batch size are the two knobs a reader would ask about next, and Table 96 sweeps both. All eight cards in this machine are RTX A6000s, so the only second device class reachable here is the CPU. It is also the unflattering comparison to make, in the sense that it is the one our own explanation predicts we will look better on: a CPU decode is closer to arithmetic-bound, so if the shortfall on the GPU is scheduling rather than arithmetic it should shrink.

Device	Rate	Batch	ms/frame rel.	ms/frame ours	fps ours	Saved %	MAC model %
A6000	q0	1	111.3	78.1	12.8	29.84	34.58
A6000	q0	2	111.1	76.7	13.0	30.97	34.58
A6000	q0	4	110.7	76.5	13.1	30.96	34.58
A6000	q32	1	111.5	84.6	11.8	24.11	28.59
A6000	q32	2	111.3	83.3	12.0	25.16	28.59
A6000	q32	4	110.8	82.9	12.1	25.17	28.59
A6000	q63	1	111.5	91.5	10.9	17.92	22.33
A6000	q63	2	111.2	89.9	11.1	19.14	22.33
A6000	q63	4	110.7	89.6	11.2	19.06	22.33
CPU, 8 thr.	q0	1	8836	5625	0.178	36.34	34.58
CPU, 8 thr.	q32	1	8153	5822	0.172	28.59	28.59
CPU, 8 thr.	q63	1	8280	6788	0.147	18.02	22.33

Table 96. Wall clock at 1080p by device class and batch size, on one exit map per rate. “rel.” is the released decoder timed in the same loop; fps is one over our per-frame time. Take from it that batch 1 costs about a point of saving against batch 4 and nothing beyond it, so the batch-1 convention this work uses is close to free; and that the A6000 falls short of the MAC model by 3.2 to 4.7 points at every rate and every batch size, while the CPU meets or beats it at the two lower rates.

results/supp_latency_batch_1920x1080.json and results/supp_latency_cpu_1920x1080.json, pinned checkpoint, 0.1 dB budget, exit maps read from results/supp_paper_curve_PAPER.json. GPU rows: 20 warm-up and 40 interleaved iterations. CPU rows: 8 threads, 1 warm-up and 5 iterations only, on a shared machine.

The CPU rows carry a noise floor and it should be read before the conclusion. The released decode does not depend on the rate, so its three timings should be equal; they range from 8.15 to 8.84 s, a spread of 8.4%, which is what 5 iterations on a shared machine buys. That is enough to support the gap at q0 and q32, where the CPU is 6.5 and 4.5 points above the A6000 on the same map, and not enough to support anything at q63, where the two agree to a tenth of a point. The direction is the one the occupancy explanation predicts, and a firmer statement needs a machine that is not shared.

B.9. What the saving is worth in joules

Board power was sampled from the driver at 100 ms, which is about the rate the sensor updates. Each condition therefore runs for a fixed wall time of 20 s rather than a fixed iteration count, so that a 30 ms decode yields two hundred power samples rather than three. Idle is measured for 4 s immediately before and after every timed run on the same card, so a neighbouring process waking up mid-measurement appears as a drift between the two rather than as a saving. Energy per frame is the mean board power over the run times the median decode time.

Condition	MACs %	Time %	Gap	Energy %	Above idle %
720p q0	34.35	30.81	3.54	30.10	29.70
720p q32	23.06	18.13	4.93	18.45	18.63
720p q63	18.02	12.99	5.03	12.96	12.94
1080p q0	32.98	29.07	3.91	29.56	29.88
1080p q32	22.88	19.14	3.75	19.34	19.48
1080p q63	17.85	14.14	3.71	14.17	14.18

Table 97. The three units side by side, all as savings of the routed decode against the released decode on the same latent. Take from it the result of this subsection: energy saved tracks time saved to within 0.5 points in every condition, while arithmetic saved runs 3.5 to 5.0 points ahead of both. Subtracting idle power changes nothing, which is the check that the agreement is not an artefact of the static draw.

results/supp_power.json, pinned checkpoint, 0.1 dB budget, NVIDIA RTX A6000 with a 300 W limit, taken under the evaluation lock. 720p is padded to 1280×768 and 15 tiles, 1080p to 2048×1280 and 40 tiles.

Condition	ms rel.	ms ours	W rel.	W ours	J rel.	J ours
720p q0	43.45	30.06	283.9	286.8	12.338	8.624
720p q32	43.50	35.61	293.7	292.6	12.774	10.418
720p q63	43.53	37.88	294.1	294.2	12.804	11.144
1080p q0	110.74	78.55	298.1	296.1	33.016	23.257
1080p q32	110.76	89.56	298.1	297.4	33.019	26.633
1080p q63	110.75	95.08	298.0	298.0	33.008	28.332

Table 98. The measurement behind Table 97, in absolute units. Take from it why energy tracks time: the routed decode draws the same board power as the released one, within 3 W in the worst case and within 1 W in four of the six conditions, and at 720p q0 it draws slightly *more*. Routing does not lower the instantaneous draw of the card. It shortens the job.

results/supp_power.json. Watts are the mean of about 200 driver samples per run; times are medians over 181 to 664 iterations. Idle over the six conditions in the order run: 74.6, 78.5, 91.2, 100.5, 106.5 and 98.6 W before, and 122.4, 120.0, 126.2, 128.4, 125.0 and 121.9 W after, on a card whose decode draw is 284 to 298 W.

The engineering result should be stated plainly: the energy saving of this method tracks its time saving, not its arithmetic saving. A partly idle GPU still draws most of its static power, so the joules follow the seconds almost exactly, and the seconds fall short of the multiply-accumulates for the occupancy reason of Figure 43c. At 1080p and the lowest rate the three read 32.98%, 29.07% and 29.56%; at the highest rate they read 17.85%, 14.14% and 14.17%. Quoting the arithmetic figure as an energy figure overstates it by three and a half to five points.

Two limits. The idle baseline drifts upward as the card warms, from 74.6 W before the first condition to 128.4 W after the fourth, so the above-idle column of Table 97 is the weaker of the two energy figures, and it is only because it agrees with the unadjusted column to within 0.4 points that the conclusion survives. Power also comes from the driver counter rather than an external meter, so the absolute joules inherit whatever bias that sensor has; the savings are ratios of two measurements taken the same way on the same card minutes apart, which is what the bias mostly cancels in.

B.10. Memory, frame rate, and what the encoder pays

Frame (padded)	Tiles	MB rel.	MB ours	Change %	fps rel.	fps ours
1024x512	8	180.0	204.0	+13.33	43.89	49.88
1280x768	15	337.5	382.5	+13.33	22.94	28.03
2048x1280	40	900.0	1020.0	+13.33	9.03	11.18
3840x2304	135	out of memory				

Table 99. Peak decoder memory and frame rate, released against ours, at every resolution that fitted. Take from it that routing costs memory rather than saving it, by a constant 13.33% at every resolution, because the tiled path holds the whole tile batch and the stitched canvas alive together; and that the frame-rate gain grows with the frame, from 43.9 to 49.9 fps at 832×480 up to 9.0 to 11.2 fps at 1080p, because a larger frame has more tiles to allocate across. The 4K row is a genuine failure and is reported as one: the job ran out of memory on a 48 GB card that four other processes were sharing, so no measurement above 1080p exists anywhere in this work.

results/supp_footprint.json, pinned checkpoint, q32, 0.1 dB budget, NVIDIA RTX A6000. Frame rates here are lower than in Table 96 because that table's maps are at different rates and this one is q32 throughout. The 4K stage profile queued alongside it failed the same way (results/supp_queue.log).

The signalled configuration moves the choice of exit map to the encoder, which has to price the tiles before it can allocate them, and that cost belongs in a complexity section even though it falls on the other side. On one 1080p sequence at the highest rate, one tiled decode takes 114.9 ms; building the per-tile cost table full-frame takes 157.9 ms, or 1.373× one decode; and building it the way the deployed encoder does, per tile, takes 519.2 ms, or 4.517×. The two searches agree on 82.5% of tiles and reach 17.39% and 16.71% saving at the same budget. The asymmetry is the design rather than an accident: a decoder-side saving is bought with encoder-side work, which suits a compress-once decode-many deployment and does not suit live encoding.

results/supp_encoder_cost_PAPER.json: pinned checkpoint, one sequence (Bosphorus), q63, 40 tiles, 15 iterations on an NVIDIA RTX A6000.

B.11. What no file in results/ pins

Four gaps, listed so that they are visible rather than quietly absent.

- **The paper's tables are on the 5C² column.** Tables 92 and 94 together say by how much: between 0.14 and 0.80 points of saving, in the optimistic direction at every exit. The hook-count column is the one a meter agrees with, and the honest ceiling is 38.33% rather than 39.1%. The BD-Rate figures are unaffected, being integrals of rate against quality with no compute axis in them; the BD-saving figures would shift by the same fraction of a point as the savings they integrate.
- **Nothing above 1080p.** Both 4K jobs ran out of memory (Table 99). GPU 2 has about 5 GB free on a card four other processes are sharing, and a 3840×2304 decode does not fit, so the resolution trend in this section rests on three points of which two carry power measurements. This is a limit of the machine and not of the method.
- **One GPU model, one power sensor, and no second card class.** Every millisecond and joule on a GPU here is an NVIDIA RTX A6000, because all eight cards in this machine are; the CPU rows of Table 96 are the only second device class, and they carry the noise floor stated beside them. Nothing here separates the decoder's energy from the board's, and no external meter was available to calibrate the driver counter against.
- **One budget for the clock.** Every timing, power and memory measurement in this section is at the 0.1 dB budget, and the stage profiles are at q32 only. The arithmetic is swept across the whole grid in Figure 43b; the seconds are not, and a deeper budget moves more tiles to exit 2, where the occupancy penalty of Figure 43c is smallest.

C. Derivations

This section states the allocation problem from its objects upward and proves what the paper asserts about it. Nothing here needs a result from elsewhere in the supplement. Each statement carries the assumptions it uses, a proof, and a remark on what it leaves open.

no.	claim	in	checked in	ckpt
P1	separates over tiles	C.2	verify_theory	eval
P2	monotone in λ	C.2	verify_theory	eval
P3	blind sets are a hull	C.3	static_RECIPES12_b01	PAPER
T4	a Minkowski average	C.3	not measured	-
P5	reaches only the hull	C.3	hull_gap	eval
P6	savings on a 1/N lattice	C.3	theory_checks	eval
P7	the floor	C.4	saturation_RECIPES12	PAPER
P8	the saturation price	C.4	verify_theory	eval
P9	the ceiling	C.4	per_class_RECIPES12	PAPER
T10	adaptivity gain	C.5	theory_check	BEST
P11	plotted log-convexity	C.6	logconvexity	none
P12	Lorenz bound	C.7	hybrid_RECIPES12_b01	PAPER
P13	rank-1 threshold rule	C.8	raterank_RECIPES12_b01	PAPER
P14	rank-1 uses hull exits	C.8	raterank_RECIPES12_b01	PAPER

Table 100. The numbered results of this section, and the evidence for each. P is a proposition and T a theorem; the numbering is stable and is what the main paper cites. PAPER is runs/RECIPES12/ckpt_PAPER.pth.tar, eval is runs/RECIPES12/ckpt_eval.pth.tar, since overwritten by the per-epoch watcher, and BEST is runs/BEST/ckpt_eval.pth.tar. verify_theory is scripts/verify_theory.py, which reports seven of seven propositions passing on one sequence at one rate.

Files named without a directory are under results/ with a .json extension; the saturation and Lorenz rows are the _ctc53 and _b01 fixed variants of the names given. T4 is proved and not measured: it describes a set of 120 vertices that no experiment enumerates.

C.1. The setting, and four assumptions

Tiles. The decoder is cut at a fixed depth. Everything above the cut runs once over the whole frame; everything below runs on square patches of the trunk’s feature grid, which are the tiles. The frame is padded by replication first, so the grid divides exactly and the tiling is a partition. A tile is 32 feature positions on a side, 16 latent positions and 256 pixels. Tiles are indexed by t and N is their number in a frame: 40 at 1080p and 2 at 416×240.

Exits. The per-tile part of the decoder is a stack of 12 residual blocks, and an exit is a point at which a tile may stop and be handed to the reconstruction head. There are $K = 6$, evenly spaced, so exit k runs $(k+1)b$ blocks with $b = 2$. The split depth is $j = 2$: blocks 0 to $jb-1$ run full-frame whatever exit a tile takes, so the usable ladder has $K - j = 4$ members and k ranges over $j, \dots, K-1$ throughout. Each exit below the deepest carries a small zero-initialised adapter, a tile leaving early handing the head a feature the head was not trained to read.

Distortion. $D_{t,k}$ is the mean squared error tile t incurs when it leaves at exit k , and two choices in that definition do real work. The error is measured on the path actually deployed, a tiled decode with replicate padding at tile borders, so what tiling costs sits inside $D_{t,k}$. And it is measured against the released decoder’s decode of the same latent, so encoder, hyperprior and entropy model are byte-identical on both sides and synthesis is the only difference. Nothing forces $D_{t,K-1}$ to zero, and C.4 returns to that.

Decibels. Distortion is reported as $10 \log_{10}$ of the ratio to the reference, and the ratio can be formed at two levels. Pooling every tile of every frame into one mean squared error is the form the Lagrangian below is exact for; averaging a per-frame decibel is the codec convention, and it is what the budget is bisected against throughout. On an identical allocation the two read 0.023 to 0.033 dB apart, 23% to 33% of the 0.1 dB budget, so a decibel here means nothing without its convention.

Convention gap from results/db_convention.json, checkpoint runs/wdec_j2_p128_grid/ckpt_epo0.pth.tar, 40 frames: an old checkpoint for a claim about arithmetic rather than about a model, and the only file in results/ measuring both conventions on one allocation. The budget section of this supplement measures the gap on the pinned checkpoint.

Compute. Compute is counted in multiply-accumulate operations and divided by those of one released full-frame decode, 453.5 GMAC at 1080p, so $c = 1$ is one released decode and a saving of $S\%$ means the

frame was decoded for $(1 - S/100)$ of one. Every operator runs at a fixed multiple of the latent grid, which makes the cost of an exit resolution-independent. One tile at exit k costs

$$c_k = s_{\text{up}} + s_{\text{trunk}} \frac{(k+1)b}{N_{\text{blk}}} + s_{\text{head}} + a_k + r \quad (40)$$

with $s_{\text{up}} = 0.0816$ the opening upsample, $s_{\text{trunk}} = 0.8944$ the trunk spread over $N_{\text{blk}}^{\text{up}} = 12$ blocks of which exit k runs $(k+1)b$, $s_{\text{head}} = 0.0240$ the head, a_k the adapter exit k wears and $r = 0.0095$ the full-frame deblocking filter. Only the trunk term depends on k , which is why a ceiling exists: however shallow the ladder gets, the stem, the head and the filter are still paid. The four shares sum to 1.0095072, the deepest-exit cost in every configuration file read here.

exit k	blocks	adapter	share	c_k	100(1-c_k)	earlier c_k
2	6	ffn	0.712	0.6088	39.12	0.5809
3	8	ffn	0.712	0.7578	24.22	0.7300
4	10	1×1	0.142	0.8697	13.03	0.8697
5	12	none	0.000	1.0095	-0.95	1.0095

Table 101. What each exit costs, in units of one released full-frame decode, with the adapter share in units of one residual block. The deepest exit costs more than 1 because a tiled decode still pays the deblocking filter. The last column is the earlier cost vector, built when the FFN adapter was billed at $2C^2$ instead of its measured $5C^2$; the checks of Propositions 5, 6, 8 and 10 were run against it and say so where they appear.

c_k from the uniform-depth rows of results/static_RECIPES12_b01.json and adapter shares from results/adapter_cost.json, both on runs/RECIPES12/ckpt_PAPER.pth.tar; earlier vector from results/tile_table.json. The complexity section audits the block count itself.

The gaps between those costs are what the allocation trades in, and they are unequal: 0.1491, 0.1119 and 0.1398. The first is largest because exit 3 gives up the FFN adapter’s saving as well as two blocks.

Assignments. An assignment is a map a from tiles to exits. There are $(K-j)^N$ of them, 4^{40} at 1080p, so they will not be enumerated. The frame’s compute and distortion are

$$C(a) = \frac{1}{N} \sum_{t=1}^N c_{a(t)}, \quad D(a) = \frac{1}{N} \sum_{t=1}^N D_{t,a(t)} \quad (41)$$

Everything after this rests on four assumptions, none of them a mathematical necessity. Each result names the ones it uses.

- **(A1) Compute is additive over tiles.** Exact when tiles decode independently, which is what a tiled decoder does, and it is why the stem and the head sit inside c_k .
- **(A2) Distortion is a tile mean**, and a tile’s error does not depend on which exits its neighbours took. The first half is exact for any per-pixel loss averaged over a frame; the second is not free, the deblocking filter running on the stitched canvas and seeing the whole exit map at once.
- **(A3) The reference is fixed.** $D_{t,k}$ is measured on the deployed tiled path against the released decoder’s own decode of the same latent, so rate is identical on both sides and cancels.
- **(A4) The cost vector is strictly increasing** in k over $k \geq j$, which Table 101 shows it is.

The second half of (A2) is checked rather than asserted. The per-tile table and the measured decode of the corresponding mixed map are recorded for 60 allocations of one frame, spanning savings of 1.2% to 41.9%. The largest disagreement between prediction and decode is 5.5×10^{-5} dB, four orders of magnitude below the budget. Nothing in (A1) to (A4) constrains $D_{t,k}$ itself, and nothing requires it to fall as k grows.

Additivity check computed from results/tile_table.json (Bosphorus, q32, 40 tiles), checkpoint runs/RECIPES12/ckpt_eval.pth.tar, since overwritten by the per-epoch watcher. What it checks is a property of the decode path rather than of the weights.

C.2. A price on compute

The encoder's problem is to spend as little compute as possible while staying inside a distortion budget:

$$\min_a C(a) \quad \text{subject to} \quad D(a) \leq D_{\text{budget}} \quad (42)$$

Constraint (42) couples the tiles: whether a tile can afford to leave early depends on how much of the budget the other 39 have used. The standard way out is to buy the constraint off with a price $\lambda \geq 0$ charged per unit of compute:

$$L(a, \lambda) = D(a) + \lambda C(a) = \frac{1}{N} \sum_{t=1}^N [D_{t,a(t)} + \lambda c_{a(t)}] \quad (43)$$

λ is an exchange rate, in squared error per unit of relative compute, so λc_k converts the compute an exit uses into the squared error that compute is deemed to be worth. Below it runs from about 2×10^{-7} to about 1×10^{-4} , against squared errors of order 1×10^{-4} on a 0 to 1 scale. What matters is where λ sits relative to the differences between exits.

At $\lambda = 0$ compute is free and every tile takes whichever exit has the smallest error. As λ grows the compute term grows fastest for the deepest exit, so a tile drops to a shallower one once the compute that exit saves, times the price, exceeds the error it adds. Past a large enough λ every tile has dropped as far as the ladder allows.

Proposition 1 (separation)

For every $\lambda \geq 0$, an assignment minimises $L(a, \lambda)$ over all assignments if and only if it minimises the bracket tile by tile:

$$k_t^*(\lambda) \in \arg \min_{k \geq j} [D_{t,k} + \lambda c_k] \quad (44)$$

Assumes (A1) and (A2).

Proof. Write $f_t(k) = D_{t,k} + \lambda c_k$. Then $N L(a, \lambda) = \sum_t f_t(a(t))$, a sum in which the t -th term depends on a only through $a(t)$. For any assignment a , term by term $f_t(a(t)) \geq \min_k f_t(k)$, so $N L(a, \lambda) \geq \sum_t \min_k f_t(k)$, and the right-hand side is attained by any a that picks a minimiser in every tile. Conversely if a is optimal and some tile had $f_t(a(t)) > \min_k f_t(k)$, replacing $a(t)$ by that minimiser and leaving every other tile alone would strictly lower L .

Remark. The result is about the frame mean and bounds no single tile. At the deployed 0.1 dB operating point the per-tile penalty over 1,765 tiles has a mean of 0.11 to 0.16 dB by rate, a 99th percentile of 0.54 to 1.18 dB and a worst tile of 1.97 dB, so a frame inside its budget can hold a tile an order of magnitude outside it. No result in this section repairs that.

Per-tile tail from results/supp_opquality_PAPER.json, checkpoint runs/RECIPES12/ckpt_PAPER.pth.tar, 53 sequences at five rates, at the operating point read from results/signalled_RECIPES12_ctc53.json.

The N -dimensional search has become N independent searches over 4 numbers each. Ties are broken toward the shallower exit, which makes k^* a function and costs nothing. We call $k_t^*(\lambda)$ the oracle's choice, because it uses the true $D_{t,k}$, which only the encoder has. The construction is Shoham and Gersho's [28] with compute in place of rate, made standard in coding by [29].

Proposition 2 (monotonicity)

Let $\lambda_1 < \lambda_2$ and let a_1, a_2 be the corresponding oracle assignments. Then $C(a_2) \leq C(a_1)$ and $D(a_2) \geq D(a_1)$. Assumes (A1), (A2) and Proposition 1.

Proof. Fix a tile and drop the index. Optimality at each price gives $D_1 + \lambda_1 c_1 \leq D_2 + \lambda_1 c_2$ and $D_2 + \lambda_2 c_2 \leq D_1 + \lambda_2 c_1$, where (D_i, c_i) is the pair chosen at λ_i . Adding the two and cancelling $D_1 + D_2$ leaves $(\lambda_2 - \lambda_1)(c_2 - c_1) \leq 0$, hence $c_2 \leq c_1$. Substituting that back into the first inequality gives $D_2 \geq D_1 + \lambda_1(c_1 - c_2) \geq D_1$. Averaging over tiles preserves both.

Remark. Monotone is not continuous. Cost moves in the jumps Proposition 6 measures, so bisecting λ converges on a reachable level

rather than on the budget. The result also assumes exact minimisation of the true table, whereas the deployed search bisects a predicted table and corrects with a real decode, at most six passes to 5×10^{-4} dB.

Monotonicity is what makes the budget searchable: bisecting λ against the measured decibels converges on the largest price whose allocation still meets the budget, and that is also the cheapest such allocation the family contains. Every λ quoted in this project is found that way, and so is the router's β .

tile	bits	D(2)	D(3)	D(4)	D(5)	exits as λ rises
34	10,446	1.608	1.638	1.679	1.655	2 throughout
0	12,996	9.346	9.249	9.207	9.180	5, 4 at 1.9, 3 at 3.0, 2 at 6.5
18	16,989	16.249	15.795	15.638	15.606	5, 4 at 2.3, 3 at 11.2, 2 at 30.6

Table 102. Three tiles of one frame, and what the price does to them. D is in units of 10^{-5} squared error and switch points in units of 10^{-6} of λ . The cheapest tile prefers the shallowest exit at every price including $\lambda = 0$, its error not falling with depth at all, so depth is not always worth buying. The most expensive tile holds out against the shallowest exit 4.7 times longer than the middle one, which is why one uniform depth cannot suit both.

results/tile_table.json (Bosphorus, q32, 40 tiles), checkpoint runs/RECIPES12/ckpt_eval.pth.tar. The 60 prices stored in that file produce 33 distinct savings, the granularity C.3 quantifies.

C.3. The achievable set, and what one price reaches

The pairs $(C(a), D(a))$ an allocation can produce form a set whose shape decides both what a sweep can reach and what adaptivity is worth. There are two versions of it. Write

$$\bar{D}_k = \frac{1}{N} \sum_{t=1}^N D_{t,k} \quad (45)$$

for the exit mean at k , the one place tiles are averaged before anything is chosen.

Proposition 3 (fixed proportions)

Suppose tiles are assigned to exits in proportions p , one probability per exit, independently of their content. Then the achievable set of expected pairs is exactly the convex hull of the K -j points whose coordinates are the cost c_k and the exit mean at k . Assumes (A1) and (A2).

Proof. Under content-independent proportions the expected cost is $\sum_k p_k c_k$ and, by the tile mean, the expected distortion is $\sum_k p_k \bar{D}_k$ times the exit mean at k . The map from p to that pair is affine and its domain is the probability simplex, whose extreme points are the unit vectors. An affine image of a simplex is the convex hull of the images of its vertices, and those images are the cost and exit mean of each exit.

Remark. The hull is a hull of expectations. One frame with N tiles realises proportions in multiples of $1/N$, so interior points are attained only to that granularity, and at $N = 2$ the smallest test class reaches three mixtures per pair of exits.

q	exit 2	exit 3	exit 4	exit 5	best single
q0	0.1791	0.0932	0.0589	0.0361	3 at 24.2%
q16	0.2194	0.1181	0.0755	0.0452	4 at 13.0%
q32	0.2819	0.1472	0.0897	0.0543	4 at 13.0%
q48	0.3330	0.1757	0.1030	0.0622	5 at -1.0%
q63	0.3955	0.2057	0.1156	0.0716	5 at -1.0%

Table 103. The vertices of the fixed-proportion set, in dB below the released decoder, one uniform-depth decode per column. These four points and their hull are everything a content-blind allocation can reach. The last column is the shallowest uniform depth that still meets a 0.1 dB budget together with its saving; at the two highest rates that is the deepest exit, which saves nothing.

results/static_RECIPES12_b01.json, checkpoint runs/RECIPES12/ckpt_PAPER.pth.tar, 53 frames. The mean over rates of the best single depth is 9.7%, against 22.1% for the per-tile allocation at the same budget.

Theorem 4 (per-tile assignment)

The convex hull of the pairs achievable by per-tile assignment is the Minkowski average of the N per-tile hulls,

$$\mathcal{A}_{\text{tile}} = \frac{1}{N} \bigoplus_{t=1}^N \text{conv}\{(c_k, D_{t,k}) : k \geq j\} \quad (46)$$

Assumes (A1) and (A2).

Proof. By (A1) and (A2) an achievable pair is the average of one point drawn from each per-tile set S_t , the set of pairs cost c_k against error $D_{t,k}$, so the achievable set is the Minkowski average of the S_t . Taking convex hulls commutes with Minkowski addition, the hull of a sum being the sum of the hulls, and with positive scaling; applying that $N-1$ times moves the hull inside the sum.

Remark. The theorem bounds what any allocation reaches, a perfect router included, and says nothing about which of those points a price reaches, which is Proposition 5. It is also the one result here with no measurement behind it: the set has up to $N(K-j-1) = 120$ vertices at 1080p and no experiment enumerates them.

Proposition 3 is the special case in which every tile has the same row. Otherwise the per-tile set is strictly larger, and the difference shows in the vertex count: the fixed-proportion hull has at most $K-j = 4$ vertices and so 3 straight segments, while the Minkowski average is smooth at any plotted scale. Figure 44 draws both on one frame.

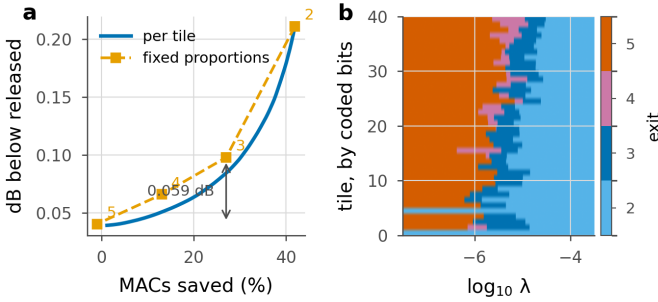


Figure 44. The two achievable sets, on one frame. **a**, the per-tile frontier of Theorem 4 traced by sweeping λ , against the fixed-proportion chain of Proposition 3 through the four uniform-depth points, labelled by exit. The per-tile curve lies below the chain everywhere between the vertices, by 0.059 dB at the exit-3 vertex. **b**, the exit each of the 40 tiles takes as λ rises, tiles ordered by the bits the entropy coder spent on them. Every row is non-increasing, which is Proposition 2 per tile, and the rows differ, which is what Theorem 4 buys over Proposition 3. The bottom row is a tile that never leaves the shallowest exit.

Both panels from results/tile_table.json (Bosphorus, q32, 40 tiles), checkpoint runs/RECIPE512/ckpt_eval.pth.tar, drawn by scripts/supp_derivations_figure.py on that file's own cost vector, the last column of Table 101. A grid of 4001 prices finds 93 distinct reachable savings on this frame, against the 95 the dynamic programme of Table 104 enumerates.

Proposition 5 (a price reaches only the lower hull)

For every $\lambda \geq 0$ the point (C, D) produced by the oracle assignment lies on the lower convex hull of the achievable set, and no λ produces a point strictly inside it. Assumes (A1), (A2) and Proposition 1.

Proof. By Proposition 1 the oracle assignment minimises $D(a) + \lambda C(a)$ over all assignments, so its pair minimises the linear functional $(C, D) \rightarrow D + \lambda C$ over the achievable set and hence over that set's convex hull. A minimiser of a linear functional over a compact convex set lies on its boundary, and because $\lambda \geq 0$ the supporting line has slope $-\lambda \leq 0$, which places the point on the lower boundary. Conversely a point strictly inside the hull is a strict convex combination of achievable points and is therefore strictly beaten, at that same C , by some point of the hull, so it cannot minimise any such functional.

Remark. A budget between two hull vertices is unreachable by any price, and the proposition does not say what that costs, which is Proposition 6. The measurement below enumerates the exact Pareto set on one frame at one rate and on the earlier cost vector.

Everett's generalised multiplier method (1963) supplies the reassuring half of the converse: whatever compute the returned allocation consumes, it is optimal for that level, so a sweep never returns something dominated. With four distinct exit costs the whole Pareto set can be enumerated by dynamic programming over tiles, so the other half can be priced exactly.

budget dB	by sweep	exact Pareto	gap
0.05	12.398	12.421	0.0230
0.08	25.443	25.489	0.0452
0.10	30.475	30.496	0.0209
0.12	33.782	33.803	0.0211
0.15	37.439	37.461	0.0211
0.18	40.048	40.046	-0.0018
0.20	41.166	41.165	-0.0016

Table 104. What convexity costs. Savings in per cent at seven budgets on one frame. The sweep reaches 95 allocations against the 635 on the exact Pareto set, and the largest loss over these budgets is 0.045 saving points. The two negative entries are the sweep landing on a budget the enumeration's grid straddles rather than the sweep beating the Pareto set, which Proposition 5 forbids.

results/hull_gap.json, Bosphorus q32, 40 tiles, on the earlier cost vector; the enumeration and the sweep use the same vector, so the gap is unaffected by the correction.

Proposition 6 (granularity)

If at most m tiles switch exit at any single breakpoint of the sweep, and each switch moves a tile by one rung, then consecutive reachable savings differ by at most

$$\text{spacing} \leq \frac{100 m \max_k (c_{k+1} - c_k)}{N} \% \quad (47)$$

and the loss from convexity cannot exceed the largest spacing. Assumes (A1), (A4) and Proposition 2.

Proof. Between breakpoints the assignment is constant, so the reachable costs are exactly the costs at the breakpoints. At one breakpoint the mean cost changes by $1/N$ times the sum of the individual changes, each at most $\max_k (c_{k+1} - c_k)$ in magnitude, and there are at most m of them; multiplying by 100 puts it in saving points. For the second part, let a budget fall strictly between two reachable levels. By Proposition 2 the bisection returns the lower level, so the loss is at most the distance between them.

Remark. Both hypotheses are read off a measured table rather than guaranteed, and the bound is vacuous if either fails on another frame. The expression falls as $1/N$, so it says nothing useful at the smallest test class, where $N = 2$ and one switch moves the frame by half the largest rung.

On the measured table $m = 2$, tiles with identical rows switching together, the largest rung is 0.1491 and $N = 40$, so the bound is 0.7453 saving points. The measured spacing is 0.7453, sitting on the bound, and the convexity loss of Table 104 is 0.0452. Nothing in the expression depends on content or on rate, and halving the tile side would put four times as many tiles on the lattice, each carrying a quarter of the step.

m , the largest rung, the bound and the measured spacing from results/theory_checks.json (Bosphorus q32, 40 tiles, runs/RECIPE512/ckpt_eval.pth.tar), written by scripts/verify_theory.py.

C.4. Floor, saturation and ceiling

The sweep has two ends, and a budget outside them buys nothing. The budget section measures where they sit at each rate; this one derives them.

Proposition 7 (the floor)

At $\lambda = 0$ the allocation attains $D_{\min} = (1/N) \sum_t \min_k D_{t,k}$, and no allocation attains less. Assumes (A2) and Proposition 1. *Proof.* Immediate from Proposition 1 at $\lambda = 0$, and from $D(a)$ being a mean of terms each bounded below by its own tile minimum.

Remark. The floor is a statement about the table, and the table is a product of training, which (A3) does not constrain. If the deepest exit drifts from the reference by δ then the achievable set shifts up by δ before any compute is saved, and a budget below δ is unreachable at every allocation. With the training objective's anchor term the drift is 0.005 to 0.036 dB over q0 to q63; without it, 0.110 to 0.159 dB, which alone consumes more than the headline budget.

Anchored drift from results/supp_anchor_PAPER.json, checkpoint runs/RECIPE512/ckpt_PAPER.pth.tar, 53 frames. Unanchored drift from results/anchor_VERBATIM.json, checkpoint runs/VERBATIM/ckpt_eval.pth.tar, 40 frames, a different training run at three rates. The two are not a controlled pair; the direction is what the argument needs and each interval is quoted as measured.

Because $D_{t,k}$ is measured on the deployed tiled path, $D_{\min}$ is not zero: it is what tiling costs before any tile exits early. Over the dense sweep the floor exceeds the budget in 3 of 45 budget-and-rate cells, all of them at 0.05 dB.

results/signalled_RECIPE512_grid.json, checkpoint runs/RECIPE512/ckpt_PAPER.pth.tar, 53 sequences, 2 frames per sequence, 9 budgets by 5 rates. Floor and saturation run 0.036 to 0.072 dB and 0.179 to 0.396 dB across the five reported rates (results/saturation_RECIPE512_ctc53.json, same checkpoint).

Proposition 8 (saturation)

Define

$$\lambda_{\text{sat}} = \max_t \max_{k>j} \frac{D_{t,j} - D_{t,k}}{c_k - c_j} \quad (48)$$

Then for every $\lambda > \lambda_{\text{sat}}$ the oracle assigns exit j to every tile, the frame's cost is exactly c_j , and no larger price changes anything. *Assumes* (A1), (A4) and Proposition 1.

Proof. Exit j beats exit k on tile t precisely when $D_{t,j} + \lambda c_j \leq D_{t,k} + \lambda c_k$, that is when $\lambda \geq (D_{t,j} - D_{t,k})/(c_k - c_j)$, the denominator being positive for $k > j$ by (A4). Taking the maximum over k and then over t gives a single price beyond which exit j wins everywhere. The cost of the constant map is c_j by (41).

Remark. λ_{sat} is a maximum over the tiles of one frame, so a price chosen for a sequence saturates some of its frames and not others, and it constrains the price rather than the budget. On the worked frame the closed form gives 3.045×10^{-5} , exactly the last switch of the most expensive tile in Table 102.

Proposition 9 (the ceiling)

The largest saving any allocation can reach is

$$S_{\max} = 100(1 - c_j) \% \quad (49)$$

Assumes (A1), (A4) and Proposition 8. *Proof.* $C(a)$ is a mean of costs each at least c_j , j being the shallowest usable exit, so $C(a) \geq c_j$ with equality only for the constant map, which Proposition 8 shows is attained.

Remark. $S_{\max} = 39.1\%$ depends on the split depth alone, so no amount of training raises it and the only lever is j . It is a ceiling in multiply-accumulates and not in seconds; the complexity section reports the wall-clock saving falling short of it. Over the 18 class-and-rate cells measured at a saturating budget the reported ceiling spreads by 6e-06 saving points, which is single-precision noise.

Per-class invariance from results/per_class_RECIPE512.json at budgets of 0.5 dB and above, checkpoint runs/RECIPE512/ckpt_PAPER.pth.tar; the ceiling itself is 39.1%, from results/saturation_RECIPE512_ctc53.json.

C.5. What adaptivity is worth

What content awareness is worth can be answered without building a predictor. Compare the best per-tile allocation with the best content-blind one at the same price:

$$J_{\text{ad}}(\lambda) = \frac{1}{N} \sum_t \min_k [D_{t,k} + \lambda c_k], \quad J_{\text{fix}}(\lambda) = \min_k [\bar{D}_k + \lambda c_k] \quad (50)$$

J_{ad} takes the minimum inside the average and J_{fix} takes it outside. By Propositions 1 and 3 the first is the best the per-tile family attains at price λ and the second the best the fixed-proportion family attains.

Theorem 10 (adaptivity gain)

For every $\lambda \geq 0$, $\Delta(\lambda) = J_{\text{fix}}(\lambda) - J_{\text{ad}}(\lambda) \geq 0$, with equality if and only if some single exit k^* attains the per-tile minimum for every tile. *Assumes* (A1) and (A2).

Proof. Fix λ and let k be any exit. Then the exit mean at k plus $\lambda c_k = (1/N) \sum_t [D_{t,k} + \lambda c_k] \geq (1/N) \sum_t \min_{k'} [D_{t,k'} + \lambda c_{k'}] = J_{\text{ad}}(\lambda)$, because each summand dominates the corresponding minimum. The right-hand side does not depend on k , so taking the minimum over k on the left preserves the inequality and gives $J_{\text{fix}} \geq J_{\text{ad}}$. For equality, suppose some k^* attains the per-tile minimum everywhere; then choosing it makes every summand equal to its minimum and the inequality is tight. Conversely suppose $\Delta = 0$ and let k^* attain J_{fix} . Then $(1/N) \sum_t ([D_{t,k^*} + \lambda c_{k^*}] - \min_{k'} [D_{t,k'} + \lambda c_{k'}]) = 0$. Every summand is non-negative, so every summand is zero, which says exactly that k^* attains the minimum on every tile.

Remark. Δ upper-bounds what any predictor gains over a content-blind allocation at that price and says nothing about whether one can get any of it, which is what C.7 measures. It is also a property of the ladder it is measured on: improving the shallow exits compresses the differences between exits, more tiles agree on the argmin, and Δ falls even as the saving rises, so it is not a figure of merit for a decoder.

Two things follow. Δ depends only on the table $D_{t,k}$ and the costs, so it is computable before any router exists. And its equality condition is a condition on the data rather than on the method: when every tile agrees on the best exit there is nothing to adapt to.

q	$\lambda=10^{-5}$	3×10^{-5}	10^{-4}	3×10^{-4}
q0	0.30%	0.39%	0.45%	0.03%
q16	0.65%	0.74%	0.38%	0.01%
q32	0.62%	1.41%	0.37%	0.00%
q48	0.62%	2.00%	0.30%	0.00%
q63	0.68%	2.09%	0.27%	0.00%

Table 105. The adaptivity gain in scale-free form, $\Delta(\lambda)$ as a percentage of $J_{\text{ad}}(\lambda)$. At the two smaller prices it rises with rate, so adaptivity is worth most where the deep blocks are doing real work. At the largest price it is near zero at every rate, which is Theorem 10's equality condition arriving: the oracle has itself collapsed onto one exit and nothing is left for a router to recover.

results/theory_check.json, checkpoint runs/BEST/ckpt_eval.pth.tar, 40 sequences (UVG, MCL-JCV and HEVC class E; classes B, C and D not measured), on the earlier cost vector of Table 101. This is the one table in the section not on the pinned checkpoint, and nothing on the pinned checkpoint currently reproduces it.

q	blind %	oracle %	gap	blind dB	shuffled dB
q0	25.38	29.90	4.52	0.1262	0.1248
q16	19.45	25.63	6.18	0.1278	0.1335
q32	15.02	20.93	5.91	0.1304	0.1411
q48	12.00	18.28	6.27	0.1373	0.1451
q63	8.05	15.62	7.57	0.1367	0.1413

Table 106. The same gap in the units the paper reports. Columns 2 to 4 are savings at a 0.1 dB budget: the best content-blind mixture of uniform depths, the per-tile oracle, and the difference. Columns 5 and 6 are decibels at the oracle's own compute, what the best blind mixture would read there and what a random shuffle of the oracle's exit map actually reads. Adaptivity is worth 4.5 to 7.6 saving points, rising with rate, against an oracle reading 0.100 dB and a shuffle reading up to 0.141 dB.

Computed from results/static_RECIPE512_b01.json, checkpoint runs/RECIPE512/ckpt_PAPER.pth.tar, 53 frames. The blind columns are the lower hull of the four uniform-depth points of Table 103 under Proposition 3, evaluated at the budget and at the oracle's compute; the shuffle column is a measured decode. At q0 the shuffle reads slightly better than the hull predicts, which bounds the interaction (A2) ignores at under 0.002 dB.

C.6. Convexity in the units the frontier is plotted in

Theorem 4 gives D convex in C . The frontier is almost never plotted in those units: it is plotted as decibels against percentage saving, where convexity is a different statement, the logarithm being concave and increasing and so preserving neither convexity nor concavity. With $S = 1 - C/c_{K-1}$,

$$dB(S) = \frac{10}{\ln 10} \ln D(C), \quad C = (1 - S) c_{K-1} \quad (51)$$

Proposition 11 (plotted convexity is log-convexity)

Let D be twice differentiable along the frontier. Then $dB(S)$ is convex if and only if D is log-convex in C :

$$DD'' \geq (D')^2 \iff \frac{d}{dC} \left(\frac{1}{\lambda} \right) \geq \frac{1}{D} \quad (52)$$

Assumes twice differentiability along the frontier, and Proposition 5 for the second form.

Proof. C is affine in S and convexity is preserved under affine reparametrisation, so by (51) dB is convex in S if and only if $\ln D$ is convex in C . Differentiating twice, $(\ln D)'' = D''/D - (D'/D)^2 = [D D'' - (D')^2]/D^2$, and $D > 0$, so $(\ln D)'' \geq 0$ if and only if $D D'' \geq (D')^2$. For the second form, the slope of the frontier is $D' = -\lambda$ by the supporting-line argument of Proposition 5, so $D'' = -\lambda'$; substituting gives $-\lambda' D \geq \lambda^2$, and dividing by $\lambda^2 > 0$ gives $-\lambda'/\lambda^2 \geq 1/D$, whose left-hand side is $d(1/\lambda)/dC$. The second form reads as a rate condition: distortion has to fall at least exponentially in compute for the plotted curve to be convex.

Remark. This is a property of the decoder rather than a consequence of Theorem 4, and a convex D can fail it. Differentiability also fails in one of the two cases: under fixed proportions D is piecewise linear with $K-j$ vertices, so $\ln D$ is piecewise concave with upward jumps at the vertices and the shape one sees depends on where one samples, whereas the per-tile hull is smooth at any plotted scale. Treating one curve as the other is how a confident claim about frontier shape becomes an artefact of vertex spacing.

q	points	condition holds	margin	quartic fit
q0	16	100.0%	0.0024	84.0%
q16	14	100.0%	0.0050	76.3%
q32	13	100.0%	0.0088	75.3%
q48	13	100.0%	0.0145	73.7%
q63	13	100.0%	0.0230	74.7%

Table 107. Testing log-convexity on the measured frontier. The direct test asks whether the secant slopes of $\ln D$ against S are non-decreasing, and the margin is the smallest increase attained. The condition holds at every point of every frontier and the margin grows tenfold from the lowest rate to the highest. Fitting a quartic and differentiating it twice instead reports 73.7% to 84.0%, and the difference is the fit rather than the data: near the ceiling the frontier is close to vertical and a polynomial overshoots, whereas the secant test has no fitted degree to choose.

results/logconvexity.json. That file records no checkpoint, so it cannot be attributed from disk; its five frontiers have 13 to 16 points each. The quartic column is the file's own quartic_frac_ok field.

C.7. How much of the oracle a partial signal recovers

The decoder cannot run the argmin of Proposition 1, $D_{t,k}$ being an error against a source it never receives. A predictor can guess it. Writing $p_{t,k}$ for a head's probability that tile t belongs at exit k , its rule is

$$\hat{k}_t = \arg \max_k [\log p_{t,k} - \beta c_k] = \arg \min_k \left[\frac{-\log p_{t,k}}{\kappa} + \lambda c_k \right] \quad (53)$$

The second form of (53) is the same rule with $\beta = \lambda \kappa$, κ being a fixed calibration turning nats into squared error. Written that way it says that a router is the oracle of (44) with a surrogate in place of $D_{t,k}$, and that one price governs both. Define the per-tile regret at a fixed price:

$$r_t(\lambda) = |D_{t,\hat{k}_t} + \lambda c_{\hat{k}_t}| - |D_{t,k^*} + \lambda c_{k^*}| \geq 0 \quad (54)$$

Non-negativity is immediate from (44). The regret is what the frame's Lagrangian loses on tile t by trusting the prediction, and by Proposition 1 those losses add. That gives the encoder an option the two extremes do not have: signal the exits of a subset S of the tiles and let the head decide the rest, which is deferral in the sense of [39, 40] with the cost of deferring made explicit. Signalling S removes exactly $\sum_{t \in S} r_t$ from the objective.

Proposition 12 (Lorenz bound)

Fix λ and let $r_{(1)} \geq r_{(2)} \geq \dots \geq r_{(N)}$ be the regrets in decreasing order with total R . For any subset S of size s , the fraction of the total regret that signalling S removes is at most

$$L(\rho) = \frac{1}{R} \sum_{i=1}^{\lceil \rho N \rceil} r_{(i)}, \quad \rho = s/N \quad (55)$$

with equality if and only if S is a set of s largest regrets. L is the Lorenz curve of the regret distribution: non-decreasing, concave, running from 0 to 1, and lying above the diagonal by an amount the Gini coefficient summarises. Assumes (A1), (A2) and a fixed λ .

Proof. The sum of any s of the $r_{(i)}$ is at most the sum of the s largest, since replacing an element of S by a larger one outside S cannot decrease the sum, and repeating that exchange terminates at a set of s largest values. Dividing by R gives the bound. Concavity holds because the increments $r_{(i)}$ are non-increasing by construction.

Remark. The bound holds at a fixed price, and the deployed system re-bisects λ at every ρ so the frame lands back on its budget. Re-bisecting enlarges the feasible set: the overridden tiles and the predicted ones then carry two multipliers, and pairs of multipliers reach allocations no single one does. The measured recovery is therefore not bounded by L , and Proposition 12 is the wrong thing to call a bound on the deployed system.

The practical content is that an unequal regret distribution can be repaired cheaply. If a tenth of the tiles carry half the regret then signalling a tenth of them removes half of what the predictor gives up, at a tenth of the bits.

q	Gini	L(.1)	rec(.1)	L(.2)	rec(.2)	L(.5)	rec(.5)
q0	0.53	35%	29%	51%	53%	87%	103%
q16	0.49	27%	33%	46%	59%	87%	102%
q32	0.61	37%	40%	59%	68%	95%	106%
q48	0.67	45%	39%	65%	63%	96%	100%
q63	0.73	53%	39%	72%	62%	99%	94%

Table 108. The Lorenz curve of the regret, and what signalling recovers. $L(\rho)$ is the share of the total regret carried by the ρ worst tiles, which Proposition 12 says is the most any signal of that size can remove at a fixed price; $rec(\rho)$ is the share of the predictor-to-oracle gap the measured sweep recovers. Regret is concentrated at every rate, Gini 0.49 to 0.73: a tenth of the tiles carries 27% to 53% of the total regret and recovers 29% to 40% of the gap. Over every reachable cell the ratio rec/L runs from 46% to 127%, for the reason the remark gives.

results/hybrid_RECIP512_b01_fixed.json, checkpoint runs/RECIP512/ckpt_PAPER.pth.tar, 53 sequences at a 0.1 dB budget; the head it is measured against was trained on runs/RECIP512/ckpt_eval.pth.tar. $rec(\rho)$ is computed here from the same file as the saving at ρ , relative to the $\rho = 0$ and $\rho = 1$ endpoints. Signalling half the tiles beats signalling all of them at 4 of 5 rates, which is the same re-bisection effect.

The signal is an entropy-coded mask over tiles plus an index per override. A fifth of the tiles costs 44 bits per frame and recovers over half the gap. The $\rho = 1$ endpoint, which reproduces the oracle's allocation, costs 100 bits over 33.3 tiles, 3.0 bits a tile against the 2 a fixed-length code over the $K-j = 4$ reachable exits would need, because at $\rho = 1$ the mask names every tile and is pure overhead. Configuration A signals its own map instead and pays 79 to 95 bits per frame, 0.021% of a typical bitrate.

Map costs from results/hybrid_RECIP512_b01_fixed.json and results/signalled_RECIP512_ctc53.json, both on runs/RECIP512/ckpt_PAPER.pth.tar.

C.8. The rate-rank rule, and the model it comes from

There is a cheaper way to guess the oracle than a trained head, and it is worth deriving rather than presenting as a heuristic, because the derivation says when it must work and how it fails. The decoder holds, before the trunk runs, the bits the entropy coder spent on each tile's latents; write b_t for that count over the frame's mean. The model is separability in log space: one scalar difficulty per tile, and one ladder profile applying to every tile in proportion.

$$\log D_{t,k} \approx u_t + \log \phi_k, \quad u_t = \alpha \log b_t + c \quad (56)$$

Equivalently $D_{t,k} = g_t \phi_k$ with $g_t = \exp(u_t) > 0$. The profile ϕ is K-j numbers fitted once per rate and α and c are two more, all three fitted leave-one-sequence-out. Nothing is learned per frame and nothing is transmitted, which is why the rule costs the decoder zero.

Proposition 13 (a rank-1 table gives a threshold rule)

If $D_{t,k} = g_t \phi_k$ with $g_t > 0$ then

$$\arg \min_k [g_t \phi_k + \lambda c_k] = \arg \min_k [\phi_k + \frac{\lambda}{g_t} c_k] \quad (57)$$

so a tile's exit depends on the tile only through the scalar λ/g_t , and by Proposition 2 the chosen exit is non-increasing in it. Sorting tiles by g_t and cutting the sorted list at K-j-1 thresholds therefore reproduces the oracle exactly. Assumes exact rank-1 structure, and Proposition 2.

Proof. Divide the bracket by $g_t > 0$, which does not move the argmin. The reduced problem is the one-tile problem of (44) with distortion ϕ and price λ/g_t , so Proposition 2 applies and its solution is non-increasing in that price. Since λ/g_t is decreasing in g_t , the exit is non-decreasing in g_t : harder tiles go deeper. The thresholds are the breakpoints of that one common problem and there are at most K-j-1 of them.

Remark. Exactness needs an exactly rank-1 table, which no real table is, and the shortfall of the deployed rule measures the departure. The proposition also says the exit is monotone in g_t and not that a bit count estimates g_t , which is a separate and weaker claim.

Proposition 14 (a rank-1 rule uses only the profile's hull)

Under the same model, an exit k is chosen for some tile at some price only if (c_k, ϕ_k) lies on the lower convex hull of the profile. Assumes exact rank-1 structure, and Propositions 5 and 13. *Proof.* By Proposition 13 the reduced problem is a single Lagrangian sweep over the points (c_k, ϕ_k) , and by Proposition 5 such a sweep reaches only hull points.

Remark. The statement is about the fitted profile rather than the true table, so an exit off the hull is dropped by the rule and may still be the oracle's choice. Hull membership is not automatic: on the single-sequence table of results/tile_table.json the same construction drops exit 4.

q	α	$\phi(2)$	$\phi(3)$	$\phi(4)$	$\phi(5)$	on hull	agree
q0	9.98	1.0242	1.0009	0.9912	0.9841	2,3,4,5	0.458
q16	4.58	1.0251	1.0021	0.9911	0.9822	2,3,4,5	0.319
q32	2.73	1.0297	1.0013	0.9893	0.9804	2,3,4,5	0.280
q48	1.94	1.0350	1.0004	0.9884	0.9771	2,3,4,5	0.350
q63	1.43	1.0494	0.9963	0.9829	0.9731	2,3,4,5	0.390

Table 109. The fitted separable model. α is the exponent relating a tile's coded bits to its difficulty and ϕ is the exit profile, both fitted leave-one-sequence-out; agree is the fraction of tiles on which the resulting rule and the oracle choose the same exit. α is positive at every rate, so a tile the entropy coder spent bits on is a tile every exit reconstructs worse, and it falls 7-fold from the lowest rate to the highest. The profile spans 4% to 8% from its shallowest entry to its deepest, which is the margin the rule works in. All four exits survive on the lower hull at every rate, so Proposition 14 costs nothing here.

results/raterank_RECIPES12_b01.json, checkpoint runs/RECIPES12/ckpt_PAPER.pth.tar, 53 sequences at a 0.1 dB budget. Hull membership computed against the shipped cost vector of Table 101. The router section reports what the rule saves against the trained head.

The rule gives up 3.4 saving points to the trained head, and the loss is not in the separability. On the measured tile table the separable model absorbs 99.96% of the variance of log D. The quantity it has to resolve is small: the exit profile spans 0.0449 in log units while the non-separable residual has a root mean square of 0.0144, 32% of it. A model can explain almost all of a table and still be unable to order the differences that decide the argmin, which reads only the differences. Predicting a tile's difficulty from its bit count alone then leaves $R^2 = 0.84$, and by Proposition 13 an error in log g_t is a proportional error in the price that tile faces. Coded bits correlate with the mean level of a tile's row at Spearman 0.83 to 0.88, with its spread at 0.61 to 0.71 and with the oracle's exit index at 0.38 to 0.54, the ordering the model predicts.

Residual analysis computed from results/tile_table.json (Bosphorus, q32, 40 tiles), checkpoint runs/RECIPES12/ckpt_eval.pth.tar; α fitted there is 4.69 against 2.73 for the 53-sequence fit at the same rate, the difference being one sequence against 53. Spearman correlations from results/raterank_RECIPES12_b01.json, pinned checkpoint; the file stores the last against the negated exit index and it is reported here with the sign flipped, so a positive value means more bits and a deeper exit.

The rule is the exact oracle for a decoder whose distortion table happens to be rank-1, so its shortfall measures how far from rank-1 the real table is, and Proposition 12 applies to it as it does to any predictor whose regret the encoder can compute.

C.9. What is proved, what is checked, and what neither covers

Table 100 lists every result with its evidence, and four gaps in it are worth naming. Theorem 10, the result the paper leans on hardest, is measured only on runs/BEST/ckpt_eval.pth.tar over 40 sequences and on the earlier cost vector; the pinned version in Table 106 gives it in saving points but not as a ratio. The convexity check of Table 107 records no checkpoint at all. The granularity and hull results of C.3 rest on one sequence at one rate, enough to check arithmetic and not enough to characterise a codec. And every proposition here concerns a frame mean under (A1) and (A2), so none bounds what a single tile gives up; the tail after Proposition 1 fills that in as a measurement rather than a guarantee.

D. Complete results

This section prints in full what the main paper samples: every class at every quality index and at three budgets, the frontier drawn class by class, and the distribution behind each mean, over sequences and over tiles. Three measurements the main paper has no room for are also here, a perceptual metric beside the decibel, the composition with weight quantisation, and the raw numbers behind the curves. It ends with the cases where the method does badly.

D.1. Every class at every rate

Class	q0	q16	q32	q48	q63
MCL-JCV	33.7	30.4	26.2	23.0	20.2
UVG	30.6	28.9	23.1	18.7	15.5
HEVC B	31.1	25.8	18.2	12.4	9.7
HEVC E	29.1	25.2	18.4	15.1	15.5
HEVC C	18.0	6.3	1.7	2.8	2.5
HEVC D	8.8	2.5	2.5	2.5	2.5
All 53, pooled	29.7	25.5	20.9	17.9	15.6
Quality given up (dB)	0.099	0.099	0.100	0.098	0.100

Table 110. Compute saved against the released decoder, by class and by quality index, at the 0.1 dB budget; the main paper prints the q0, q32 and q63 columns of the first six rows. Read it in two directions. Along a row the saving falls with rate, because the residual a shallow exit fails to reconstruct grows relative to the quality being protected. Down a column it falls with resolution, from 33.7% on MCL-JCV to 8.8% on HEVC D at q0, because a 256 px tile is 40 tiles on a 1080p frame and two at 416x240. The last row is the quality the pooled allocation gave up, which is the budget to within a thousandth of a decibel at every rate.

results/supp_per_class_budgets.json, on runs/RECIPE512/ckpt_PAPER.pth.tar, 53 sequences at one frame each, per-frame convention. The pooled row reads 29.7% at q0 where the main paper's 29.9% comes from results/signalled_RECIPE512_ctc53.json; the two bisect the same budget with different code and settle on $\lambda = 5.126 \times 10^{-5}$ and 5.150×10^{-5} .

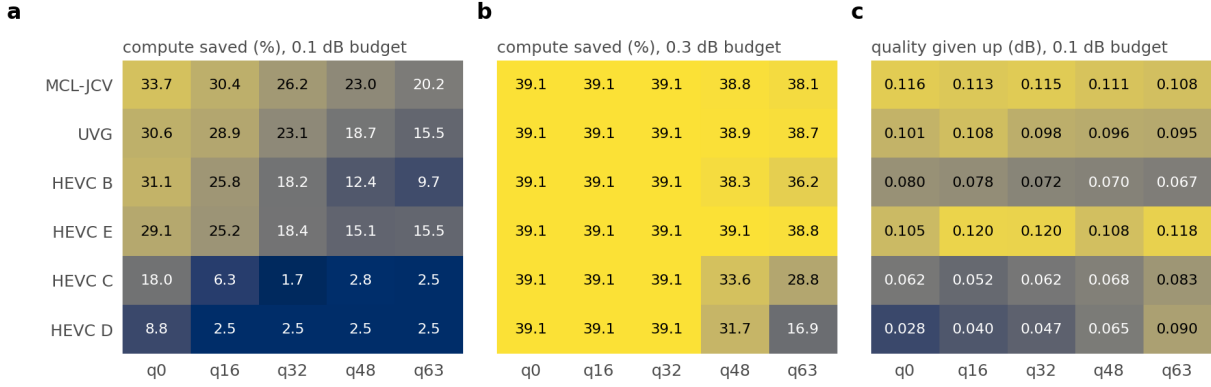


Figure 45. The same measurement as a grid, with the looser budget beside it. **a**, compute saved at the 0.1 dB budget. **b**, the same at 0.3 dB, where every class at the three lowest rates has reached the architectural ceiling of 39.1% and only q48 and q63 are still making a choice. **c**, the quality actually given up at the 0.1 dB budget, which the set meets and no class does: at q0 the allocation spends 0.116 dB on MCL-JCV and 0.028 dB on HEVC D. The small classes are cheap in quality for the same reason they are poor in saving, that with two or eight tiles there is no fine-grained way to spend a budget. The 0.5 dB grid is not drawn because every one of its thirty cells is at the ceiling.

Drawn by scripts/supp_results_figs.py from results/supp_per_class_budgets.json, on runs/RECIPE512/ckpt_PAPER.pth.tar.

The resolution effect is a property of the tiling and not of the content, and it is the argument against reading a single mean over a test set that mixes resolutions. MCL-JCV is 30 of the 53 sequences and all of them are 1080p, so it carries most of the pooled row.

D.2. Where the tiles exit

Rate	e2	e3	e4	e5	Blocks skipped	Saved
q0	67.4	20.7	8.3	3.6	5.04	29.7
q16	58.2	18.6	13.1	10.0	4.50	25.5
q32	43.5	19.5	18.8	18.2	3.77	20.9
q48	34.4	16.8	24.3	24.5	3.22	17.9
q63	22.3	23.1	28.1	26.6	2.82	15.6

Table 111. The exit histogram at the 0.1 dB budget, as a percentage of the 1765 tiles in one pass over the test set, with the mean number of trunk blocks skipped and the saving that buys. At q0 two thirds of tiles take the shallowest selectable exit and the ladder above it is barely used. At q63 the mass is spread almost evenly over the four exits and the mean depth skipped has fallen from 5.04 blocks to 2.82. Exits e0 and e1 are omitted because the exit clamp cannot select them: the split depth is 2, so a map naming a shallower exit runs to e2 and wears e2's adapter.

Histograms summed over the six classes of results/supp_per_class_budgets.json, on runs/RECIPE512/ckpt_PAPER.pth.tar. Blocks skipped per exit from results/adapter_cost.json, same checkpoint.

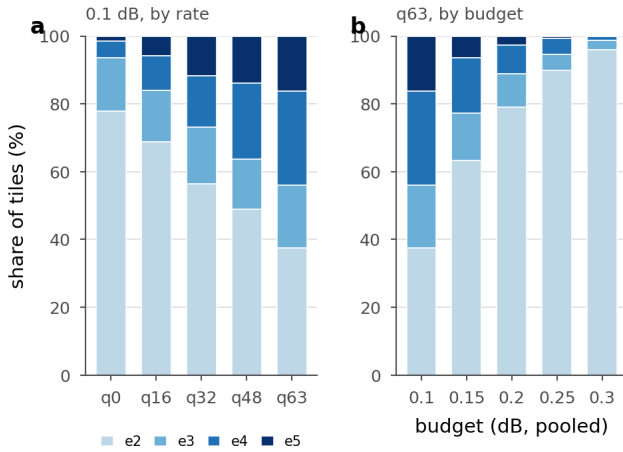


Figure 46. How the ladder fills up. **a**, exit shares at the 0.1 dB budget as the rate rises. **b**, exit shares at q63 as the budget loosens, until at 0.3 dB 96% of tiles are already at the shallowest exit and there is almost nothing left to allocate. Both panels are pooled over the test set.

Drawn by scripts/supp_results_figs.py from results/supp_paper_curve_PAPER.json, on runs/RECIPE512/ckpt_PAPER.pth.tar, pooled convention, so the budgets on the horizontal axis of **b** are looser than the identically labelled ones in the table above.

Class	e2	e3	e4	e5	e2	e3	e4	e5
MCL-JCV	73.8	17.2	5.7	3.4	29.8	23.4	23.5	23.3
UVG	54.3	31.1	14.3	0.4	5.7	31.4	44.6	18.2
HEVC B	64.0	18.5	12.5	5.0	6.5	11.0	37.5	45.0
HEVC E	51.1	35.6	4.4	8.9	15.6	35.6	8.9	40.0
HEVC C	3.1	53.1	31.2	12.5	0.0	0.0	25.0	75.0
HEVC D	0.0	25.0	25.0	50.0	0.0	0.0	25.0	75.0

Table 112. The same histogram split by class at the two ends of the rate range: the first four columns are q0 and the second four are q63, each as a percentage of that class's tiles. HEVC C and HEVC D reach neither of the two shallowest exits at q63, and three quarters of their tiles run to full depth, which is the 2.5% saving of the first table seen tile by tile. UVG behaves differently from MCL-JCV at q63 despite being the same resolution, which is content and not geometry.

results/supp_per_class_budgets.json, on runs/RECIPE512/ckpt_PAPER.pth.tar.

D.3. The frontier, class by class

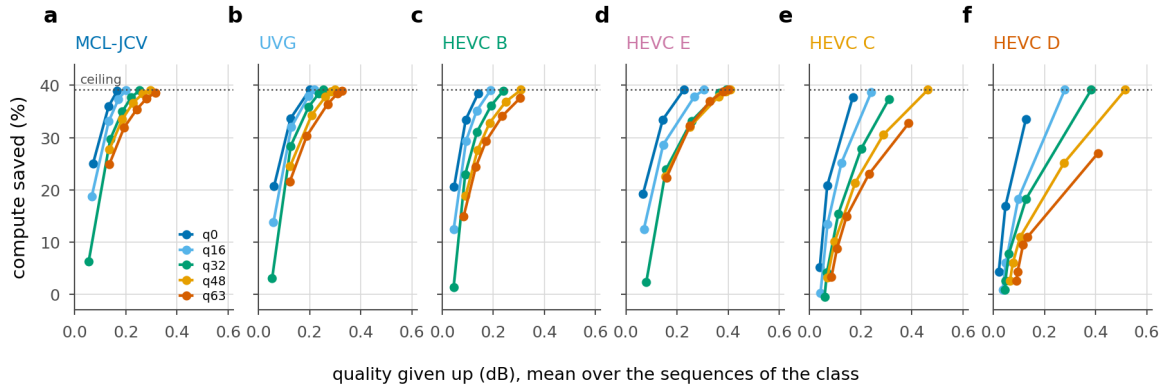


Figure 47. Compute saved against quality given up, one panel per class and one curve per quality index, from the seven budgets at which the frontier was measured. The dotted line is the architectural ceiling of 39.1%. Three things are visible that a table of operating points hides. The curves are steep and then flat, so most of the saving is bought in the first tenth of a decibel and the rest of the band buys little. The rate ordering is preserved at every budget, so a curve never crosses another within a panel. And the panels move to the right as resolution falls: MCL-JCV reaches the ceiling by 0.3 dB at every rate, while HEVC D needs 0.5 dB at q48 and does not reach it at q63 within the measured range.

Drawn by scripts/supp_results_figs.py from results/supp_paper_curve_PAPER.json, on runs/RECIPES12/ckpt_PAPER.pth.tar. A point is the mean over the sequences of that class at a global operating point, so the multiplier is the one the whole set was bisected to and not a per-class retuning. Class membership per sequence from results/supp_opquality_PAPER.json, same checkpoint.

Two limits bound every panel. A budget below the floor cannot be met at all, because our deepest exit already differs from the released decoder by 0.036 dB at q0 and 0.072 dB at q63; a budget above 0.179 dB at q0 or 0.396 dB at q63 changes nothing, because every tile is already at the shallowest selectable exit. Between those two the multiplier is doing work, and the 0.1 dB budget is inside the window at every rate.

D.4. Per-sequence spread

A set mean is not what any single clip receives. One multiplier prices compute for the whole set, so each sequence lands wherever its own content puts it, and the spread of those landings is what a deployment would have to plan against.

Rate	n	Mean	SD	Min	p25	Median	p75	Max
q0	53	32.7	7.5	6.0	28.8	35.9	38.8	39.1
q16	53	28.8	10.6	-1.0	23.6	31.7	37.6	39.1
q32	53	24.6	12.0	-1.0	18.2	26.1	34.3	39.1
q48	53	22.4	11.7	-1.0	13.8	23.5	31.4	39.1
q63	53	20.0	11.1	-1.0	12.7	20.4	27.7	39.1

Table 113. Compute saved per sequence at the 0.1 dB budget: the distribution behind the mean, over all 53 sequences. The interquartile range widens from 9.9 points at q0 to 15.0 at q63, and the gap between best and worst sequence widens from 33.1 to 40.1. The minimum is negative at four of the five rates, because a sequence whose tiles all run to full depth costs 0.95% more than the released decoder: our deepest exit is the more expensive of the two. Counting over rates as well as sequences, 33 of the 265 pairs save less than a tenth of a decade at this budget.

results/supp_per_sequence_PAPER_b010.json, on runs/RECIPES12/ckpt_PAPER.pth.tar, read from its rows_vs_release block so that the denominator is the released decoder. Pooled convention, because the operating points come from results/supp_paper_curve_PAPER.json.

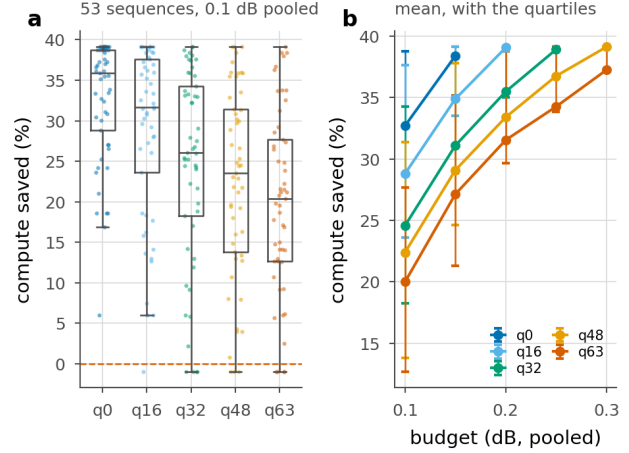


Figure 48. The same distribution drawn. **a**, every sequence at the 0.1 dB budget, one point per sequence, with the quartiles and whiskers over them; the dashed line is the released decoder. **b**, the mean with its quartiles as the budget loosens. A rate disappears from **b** once its frontier saturates, because a saturated operating point has no allocation left to break down.

Drawn by scripts/supp_results_figs.py from the five results/supp_per_sequence_PAPER_b0*.json files, all on runs/RECIPES12/ckpt_PAPER.pth.tar.

Sequence	q0	q63	q63 dB
RaceHorses_832x480	18.6	-1.0	0.094
BlowingBubbles_416x240	18.6	-1.0	0.075
BQSquare_416x240	6.0	-1.0	0.185
RaceHorses_416x240	18.6	-1.0	0.086
PartyScene_832x480	16.9	2.5	0.110
BQMall_832x480	26.5	5.7	0.107
videoSRC27	39.1	38.4	0.210
videoSRC29	39.1	39.1	0.194

Table 114. The six hardest sequences at q63 and the two easiest, with their q0 value beside them and the decibel each actually received. All six of the hard ones are 832x480 or 416x240. The delivered decibel is not the budget on any of them: across the 53 sequences at q63 it runs from 0.013 dB to 0.237 dB against a 0.1 dB target, so a single multiplier for the set means some clips overshoot the budget by more than a factor of two while others use an eighth of it.

results/supp_per_sequence_PAPER_b010.json, on runs/RECIPES12/ckpt_PAPER.pth.tar. Savings are against the released decoder; the decibel is the sequence's own, in the pooled convention.

D.5. A second metric, and the tiles that pay for the mean

The whole mechanism is denominated in PSNR, and PSNR is known not to track visual quality closely. Two things follow that are worth measuring rather than asserting: whether a perceptual metric agrees that

the loss is small, and how the loss is distributed over tiles, since a mean of 0.1 dB is compatible with a few tiles losing a great deal.

Rate	bpp	PSNR	ours	lost	MS-SSIM	ours	lost
q0	0.2313	32.536	32.435	0.1018	0.89934	0.89813	0.0520
q16	0.2671	35.335	35.235	0.1001	0.94904	0.94850	0.0455
q32	0.3352	38.135	38.035	0.1004	0.97565	0.97542	0.0399
q48	0.4561	40.777	40.674	0.1026	0.98790	0.98779	0.0376
q63	0.6717	43.155	43.037	0.1181	0.99419	0.99413	0.0464

Table 115. PSNR and MS-SSIM at the 0.1 dB operating point, over the whole test set. MS-SSIM agrees with PSNR about the direction and is kinder about the size: expressed as $-10\log_{10}(1 \text{ minus MS-SSIM})$, the loss is 0.052 dB at q0 against 0.102 dB of PSNR, and 0.046 against 0.118 at q63, so on this metric the budget costs about half what the decibel it is written in suggests. Each metric is given as the released decoder, ours, and what was lost, with the MS-SSIM loss expressed as $-10\log_{10}(1 \text{ minus MS-SSIM})$ so that the two loss columns are in the same unit. The bpp column is the released bitstream, which early exiting does not change beyond the exit map.

results/supp_opquality_PAPER.json, on runs/RECIPE512/ckpt_PAPER.pth.tar, 53 sequences at one frame each. MS-SSIM is Wang et al. 2003 at five scales with an 11-tap Gaussian of $\sigma = 1.5$ on the valid region, on the luma plane in 4:2:0 on 0 to 255, which is the domain the PSNR column uses. The implementation self-checks at start-up and the file records both results: identical inputs score 1.000000 and a 3x3 box blur scores 0.7797. λ is read from results/signalled_RECIPE512_ctc53.json so the allocation is the deployed one, and the delivered decibel recomputed here matches the stored value to four decimals at every rate.

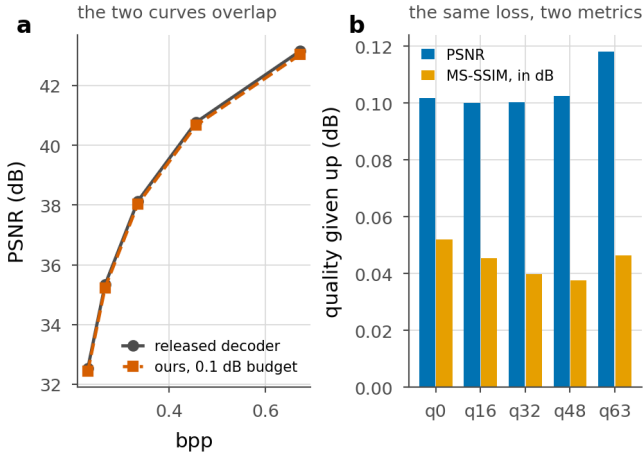


Figure 49. a, the rate-quality curve of the released decoder and of ours at the 0.1 dB budget, over the five quality indices. At plot scale they are one curve, which is what a 0.1 dB budget means. b, the gap that panel a cannot show, in PSNR and in MS-SSIM expressed as a decibel.

Drawn by scripts/supp_results_figs.py from results/supp_opquality_PAPER.json, on runs/RECIPE512/ckpt_PAPER.pth.tar.

Rate	Tiles	Mean	Median	p95	p99	Max	Over 0.25	Over 0.5	Over 1.0
q0	1765	0.164	0.109	0.463	1.184	1.971	302	79	26
q16	1765	0.153	0.106	0.480	0.852	1.495	267	80	7
q32	1765	0.142	0.104	0.400	0.671	1.587	250	50	2
q48	1765	0.151	0.100	0.441	0.934	1.636	237	74	17
q63	1765	0.111	0.098	0.288	0.539	1.264	136	21	4

Table 116. How the 0.1 dB is distributed over the 1765 tiles of one pass over the test set. The last three columns count tiles losing more than a quarter, a half and a whole decibel. The median tile is at or below the budget at every rate, and the tail is long: at q0, 302 tiles lose more than 0.25 dB, 79 lose more than 0.5 dB, and the worst loses 1.97 dB. A budget stated as a mean says nothing about the last three columns, which is why they are printed.

results/supp_opquality_PAPER.json, on runs/RECIPE512/ckpt_PAPER.pth.tar, at the same operating points as the metric table above. A tile penalty is the decibel between the released decode of that tile and ours.

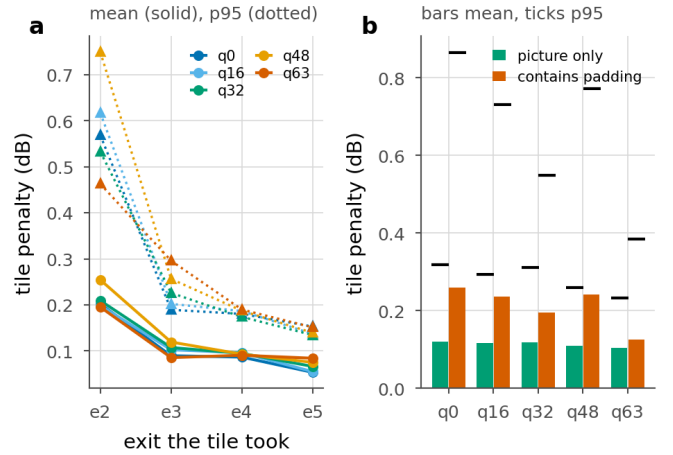


Figure 50. Where the tail comes from. a, mean and 95th percentile tile penalty against the exit the tile took. The tail sits on e2, the shallowest exit the clamp allows: at q0 its mean is 0.202 dB against 0.054 dB at e5, and all 79 tiles above 0.5 dB took it. b, the same penalty split by whether the tile contains replicated padding. A 1080p frame is padded to 2048x1280, so 200 of the bottom tile row's 256 rows are invented by replication, and those tiles carry roughly double the mean penalty at every rate: 0.261 dB against 0.121 dB at q0, and 65 of the 79 tiles above half a decibel are padded ones.

Drawn by scripts/supp_results_figs.py from the tail_by_exit and tail_by_padding blocks of results/supp_opquality_PAPER.json, on runs/RECIPE512/ckpt_PAPER.pth.tar. A tile counts as padded if any of its pixels came from the replicate pad, recorded per tile as pad_fraction. Padding geometry from results/tile_definition.json, same checkpoint.

D.6. Composition with weight quantisation

Early exiting removes work and quantisation makes the remaining work cheaper, so the two ought to compose. Whether they do is a question about the same weights, and it is measured here rather than assumed.

Weights	Rate	e2	e3	e4	e5	Layers
float32	q0	0.172	0.083	0.052	0.027	0
float32	q32	0.381	0.212	0.133	0.048	0
float32	q63	0.890	0.545	0.344	0.108	0
int8	q0	0.178	0.088	0.057	0.032	86
int8	q32	0.407	0.237	0.159	0.072	86
int8	q63	1.001	0.660	0.463	0.223	86
int6	q0	0.300	0.206	0.166	0.135	86
int6	q32	1.021	0.845	0.735	0.633	86
int6	q63	2.830	2.568	2.407	2.183	86
int4	q0	1.651	1.525	1.484	1.400	86
int4	q32	4.228	4.026	4.008	3.892	86
int4	q63	8.177	7.850	7.929	7.830	86

Table 117. Quality given up against the released decoder with the decoder weights quantised, at four exits and three rates. The float32 rows are the ladder itself and are the baseline each integer row should be read against. At q0 int8 is nearly free, moving the deepest exit from 0.027 dB to 0.032 dB, so the two levers compose. At q63 they compete: the deepest exit moves from 0.108 dB to 0.223 dB, which is more than a whole 0.1 dB budget spent before any tile has exited early. Six-bit costs between 0.63 and 2.18 dB at the deepest exit and four-bit is unusable at every rate. The last column is the number of layers quantised, the same 86 in every integer row.

results/supp_quant_PAPER.json, on runs/RECIPE512/ckpt_PAPER.pth.tar, 32 held-out images at 512 px. Weight-only, symmetric, per output channel, no calibration, applied to dec.* and router_head.* with the encoder asserted unchanged. This supersedes results/quant_BEST.json, the same experiment on runs/BEST/ckpt_eval.pth.tar. Nothing here measures an integer kernel: the file records the bit width and the implied bit-operation ratio, and the arithmetic was carried out in floating point on quantised weights.

D.7. Integrated figures, and what they depend on

A budget is one sample of a curve. Video coding answers this for rate against quality with Bjontegaard's construction [4], integrating one axis over a stated interval of the other, and the same construction applies with

compute in place of rate. BD-saving is the mean compute saved over an interval of decibels and BD-quality is the mean decibel paid over an interval of saving. Neither means anything without its interval and its convention, and both are given with theirs.

Rate	BD-saving	BD-quality	BD-saving	BD-quality	Points
q0	n/a	0.0544	n/a	0.0453	15
q16	n/a	0.0686	n/a	0.0576	14
q32	n/a	0.0861	n/a	0.0721	13
q48	31.1%	0.0981	32.8%	0.0816	13
q63	28.7%	0.1121	30.8%	0.0923	13
Mean where defined	29.9%	0.1051	31.8%	0.0870	

Table 118. BD-saving and BD-quality per rate under both decibel conventions: the first pair of columns is per frame and the second is pooled. BD-saving is defined at only two of the five rates, because at q0, q16 and q32 the frontier saturates below the upper limit of the interval and there is no curve left to integrate. The convention moves the answer by 1.9 points of BD-saving and 0.018 dB of BD-quality on identical allocations, and it moves it consistently, since per-frame averaging raises every floor and pooling does not. Points is the number of measured frontier points inside the interval.

results/supp_bd_PAPER_per_frame.json and results/supp_bd_PAPER_pooled.json, both on runs/RECIPE512/ckpt_PAPER.pth.tar over 53 sequences, integrating results/supp_paper_curve_PAPER.json. BD-saving is over 0.066 to 0.300 dB per frame and 0.059 to 0.300 dB pooled, the lower limit being the highest floor any rate has under that convention. BD-quality is over 10% to 30% saving. The floors that set those lower limits are recorded per rate in both files as floor_db, and appear directly in the frontier table below at the two rates where the 0.05 dB budget is unreachable.

Two sensitivities sit around those numbers and are worth stating beside them. The first is the anchor. Our deepest exit is not bit-exact with the released decoder, since it carries seam repair and a fine-tuned trunk, and it starts every budget slightly behind: the drift runs from 0.005 dB at q0 to 0.036 dB at q63, which at q63 is a third of the whole budget. Setting it to zero would raise BD-saving at q63 from 28.7% to 34.9%, a gain of 6.2 points. The second is the interval. Over six defensible intervals the mean BD-saving moves from 22.8% to 29.2%, a range of 6.3 points, while the gap between the lowest and highest rate stays inside 15.3 to 16.8 points on every one of them; the level depends on the integration and the rate effect does not.

Anchor drift from results/supp_anchor_PAPER.json, on runs/RECIPE512/ckpt_PAPER.pth.tar, 53 frames, per-frame convention; the zero-drift figure is the bd_saving_pct_zero_drift field of results/supp_bd_PAPER_per_frame.json. Interval sensitivity from results/bd_sensitivity.json, which is not pinned: scripts/bd_sensitivity.py names its curve as results/paper_curve_grid128.json, on runs/wdec_j2_p128_grid/ckpt_epo0.pth.tar at 128 px tiles over 40 sequences, so its levels are not the levels above and only its robustness conclusion is quoted.

Budget	A BD-Rate	B BD-Rate	Difference	A saved	B saved
0.1 dB	1.034%	1.006%	0.028 pt	22.1%	17.6%
0.3 dB	2.813%	2.795%	0.018 pt	38.2%	35.6%
0.5 dB	3.075%	3.055%	0.021 pt	39.1%	39.0%

Table 119. BD-Rate cost of the two configurations: the rate a reader would have to spend to buy back the quality the compute saving costs, integrated over the five rate points, which is the number that makes this trade-off comparable with a published codec result. Configuration A signals the exit map in the bitstream and pays between 79 and 95 bits a frame for it, configuration B infers it from what the decoder already holds and pays none. Their BD-Rate costs differ by two hundredths of a point at every budget, so the map is close to free; what A buys with it is compute, and the gap in the last two columns is 7.7 points at the tight budget and vanishes at the loose one.

results/bd_rate.json, every row on runs/RECIPE512/ckpt_PAPER.pth.tar, which its sources block records file by file. The file it replaced is kept as results/bd_rate_before_pinning.json, whose saving column reads 41.9% at 0.5 dB where the pinned one reaches 39.1%.

D.8. Raw values behind the curves

Everything above is an integral, a mean or a picture. What follows is the measured numbers themselves, so that a reader can compare against them without digitising a plot or rerunning a decode.

Rate	Budget	lambda	dB	Model	Meter	Map bits
q0	0.1	5.150e-05	0.0998	29.9	29.2	79
q0	0.2	1.000e+00	0.1791	39.1	38.3	48
q0	0.3	1.000e+00	0.1791	39.1	38.3	48
q0	0.5	1.000e+00	0.1791	39.1	38.3	48
q16	0.1	2.280e-05	0.1000	25.6	25.0	86
q16	0.2	1.284e-04	0.1999	37.8	37.0	52
q16	0.3	1.000e+00	0.2194	39.1	38.3	48
q16	0.5	1.000e+00	0.2194	39.1	38.3	48
q32	0.1	9.080e-06	0.1000	20.9	20.4	93
q32	0.2	4.135e-05	0.1999	33.7	33.0	64
q32	0.3	1.000e+00	0.2819	39.1	38.3	48
q32	0.5	1.000e+00	0.2819	39.1	38.3	48
q48	0.1	4.731e-06	0.1000	18.3	17.9	94
q48	0.2	2.061e-05	0.1997	31.7	31.0	71
q48	0.3	6.974e-05	0.2993	37.9	37.1	52
q48	0.5	1.000e+00	0.3330	39.1	38.3	48
q63	0.1	2.874e-06	0.1000	15.6	15.2	95
q63	0.2	1.103e-05	0.1999	29.6	28.9	78
q63	0.3	3.221e-05	0.2999	35.8	35.0	57
q63	0.5	1.000e+00	0.3955	39.1	38.3	48

Table 120. The deployed operating points: four budgets at five rates. Model is the saving the cost model predicts for the chosen map and Meter is what a forward hook on every convolution and linear layer counted on the same decode, so the two columns are the cost model checked against itself; the model reads high by 0.4 to 0.8 points throughout. Map bits is the entropy of the per-frame exit histogram, charged to the bitstream before any saving is computed, and it falls as the budget loosens because the histogram concentrates.

results/signalled_RECIPE512_ctc53.json, on runs/RECIPE512/ckpt_PAPER.pth.tar, 53 sequences, per-frame convention. λ is the multiplier the two-level bisection settled on and the saving is against the released decoder.

Rate	Budget	lambda	dB pooled	dB per frame	Saved	e2/e3/e4/e5
q0	0.05	2.065e-05	0.0500	0.0610	23.2	36/27/20/17
q0	0.1	7.286e-05	0.0999	0.1171	34.9	78/16/5/2
q0	0.15	2.371e-04	0.1499	0.1686	38.8	98/2/0/0
q0	0.2	n/a	0.1597	0.1791	39.1	100/0/0/0
q0	0.25	n/a	0.1597	0.1791	39.1	100/0/0/0
q0	0.3	n/a	0.1597	0.1791	39.1	100/0/0/0
q0	0.5	n/a	0.1597	0.1791	39.1	100/0/0/0
q16	0.05	6.631e-06	0.0500	0.0594	16.6	24/20/20/35
q16	0.1	3.199e-05	0.1000	0.1179	31.9	69/15/10/6
q16	0.15	7.234e-05	0.1499	0.1673	36.9	88/9/2/1
q16	0.2	2.483e-04	0.1998	0.2167	39.1	100/0/0/0
q16	0.25	n/a	0.2022	0.2194	39.1	100/0/0/0
q16	0.3	n/a	0.2022	0.2194	39.1	100/0/0/0
q16	0.5	n/a	0.2022	0.2194	39.1	100/0/0/0
q32	0.05	9.305e-07	0.0500	0.0544	5.0	6/8/11/75
q32	0.1	1.398e-05	0.1000	0.1212	28.0	56/17/15/12
q32	0.15	2.862e-05	0.1500	0.1716	34.2	78/12/8/3
q32	0.2	5.497e-05	0.1999	0.2199	37.4	91/7/2/0
q32	0.25	1.445e-04	0.2498	0.2741	39.0	99/1/0/0
q32	0.3	n/a	0.2581	0.2819	39.1	100/0/0/0
q32	0.5	n/a	0.2581	0.2819	39.1	100/0/0/0
q48	0.05	n/a	0.0544	0.0599	n/a	none
q48	0.1	7.190e-06	0.1000	0.1204	25.5	49/15/22/14
q48	0.15	1.508e-05	0.1500	0.1728	32.3	71/12/11/5
q48	0.2	2.682e-05	0.1998	0.2228	36.0	85/8/6/1
q48	0.25	4.950e-05	0.2500	0.2768	38.1	95/4/1/0
q48	0.3	1.548e-04	0.2996	0.3322	39.1	100/0/0/0
q48	0.5	n/a	0.3005	0.3330	39.1	100/0/0/0
q63	0.05	n/a	0.0584	0.0659	n/a	none

Rate	Budget	lambda	dB pooled	dB per frame	Saved	e2/e3/e4/e5
q63	0.1	4.293e-06	0.1000	0.1225	22.6	38/18/28/16
q63	0.15	8.642e-06	0.1500	0.1757	30.2	63/14/16/6
q63	0.2	1.500e-05	0.1999	0.2274	34.4	79/10/9/3
q63	0.25	2.537e-05	0.2499	0.2709	36.9	90/5/5/1
q63	0.3	4.557e-05	0.2998	0.3333	38.4	96/3/1/0
q63	0.5	n/a	0.3534	0.3955	39.1	100/0/0/0

Table 121. The frontier itself: every operating point behind the per-class figure, at seven budgets and five rates, with the exit shares that produced each one. Both decibel conventions are printed for the same allocation and they differ by 0.02 to 0.04 dB throughout, a quarter to a third of a 0.1 dB budget. A row reading n/a in the saving column is a budget below the floor at that rate, where no allocation meets the target; a row at 0.5 dB is past saturation, where the file stores no histogram because the allocation has stopped changing.

results/supp_paper_curve_PAPER.json, on runs/RECIPE512/ckpt_PAPER.pth.tar. Saving is converted to the released-decoder denominator by charging the retained cost the price of our deepest exit, 1.0 released decodes, derived here from the ceiling in results/saturation_RECIPE512_ctc53.json and the saturated saving in this file. One caution about that denominator: this file bills the shared stem inside every tile's cost, while results/supp_latency_batch_1920x1080.json bills it once per frame through flexuf/cost.py frame_relative_cost, and the two accountings differ by about a third of a point on an identical exit map at q0.

D.9. Failure cases

Five cases are worth naming, all of them measured in the files above rather than inferred.

- **Sequences that cost more than they save.** At q63 four sequences finish at minus 0.95%, which is every tile at full depth and our deepest exit charging 1.0 released decodes for the privilege: RaceHorses at 832x480 and at 416x240, BlowingBubbles and BQSquare. Three more sit under 3%. On this content the method should be switched off rather than run at a budget, and the exit histogram is what tells the encoder so.
- **The bottom tile row of a padded frame.** The six worst tiles in the test set are all in the last row of a 1080p frame at 78% padding, each losing between 1.72 and 1.97 dB. The allocation is behaving correctly given its table, and the table is measuring a region that will be cropped away before anyone looks at it, so the budget is being spent on invented pixels.
- **The two smallest classes at high rate.** HEVC C and HEVC D save 2.5% at q63, which is one tile in four moved one exit up and nothing else. With two tiles on a 416x240 frame there is no allocation worth making, and the honest reading is that this method needs a frame large enough to hold a useful number of tiles.
- **Quantisation and early exit compete at high rate.** Int8 weights cost 0.115 dB at q63 at the deepest exit, more than the whole budget the allocation is then asked to work inside, while at q0 they cost 0.005 dB. A deployment cannot assume the two savings multiply.
- **A 0.5 dB budget has no per-sequence breakdown to report.** The queue that produced this section asked for one and the job failed. The cause is the result: at 0.5 dB every rate is past saturation, the frontier stops at the ceiling of 39.1%, and a saturated operating point carries no allocation to break down. The sweep now runs from 0.10 dB to 0.30 dB.

Worst tiles from the worst_tiles block of results/supp_opquality_PAPER.json and negative sequences from results/supp_per_sequence_PAPER_b010.json, both on runs/RECIPE512/ckpt_PAPER.pth.tar. The failed job is recorded in results/supp_queue2.log at 12:06:04, and the saturation it ran into in results/saturation_RECIPE512_ctc53.json.

D.10. What this section does not measure

- No interval on any headline comes from a repeated training run. No configuration in this work was trained twice and no seed is set in the decoder trainer, so the spread reported above is across sequences at a fixed checkpoint and not across runs. Evaluation is deterministic once a checkpoint is fixed and training is not, which matters when reading

any comparison of a few tenths of a point.

- The 0.1 dB budget is a convention this work adopts and not a perceptual threshold. Subjective work measures the smallest noticeable change in a quantisation parameter or in a video quality metric, not in tenths of a decibel of PSNR, so no published result licenses a claim that 0.1 dB is invisible. The MS-SSIM table is evidence that the loss is small on a second metric, and the 0.3 dB and 0.5 dB columns are there so that nothing rests on the choice.
- One integrated figure is still not pinned. The interval study is on a 128 px run over 40 sequences and is quoted only for its robustness conclusion; every other table in this section is on runs/RECIPE512/ckpt_PAPER.pth.tar.
- Every measurement here is on intra frames of video sequences at one frame per sequence. Nothing here measures the inter-frame path, a second decoder, a resolution above 1080p, or a traditional-codec anchor. The first three are out of reach on the evaluation card, which has about 5 GB free; the last is a scope decision, since the axis this work measures is decoder-side compute against a fixed learned decoder whose own position against VTM is published.

E. The operating window

A budget D is the peak-signal-to-noise ratio the decode is allowed to give up, and an allocation is admissible if the loss it incurs stays at or below it. Between a floor, below which no allocation is admissible, and a saturation point, above which every allocation has reached the same ceiling, lies the window of budgets in which the decoder has anything to trade. Section C derives the two ends and Section D tabulates them; this section is about the window itself: how the two ends are measured and how far the measurements agree, what has to be stated with every decibel before any of it means anything, the sweep across the whole window, and how much of the rescaling that collapses the five rates onto one curve survives a change of sweep, of convention or of checkpoint.

E.1. Floor, saturation point and ceiling

The floor $D_{\min}$ is the loss of a tiled decode with every tile at full depth. No early exiting has happened, so it is the tiling penalty alone: the border of every tile has been convolved against padding rather than against its neighbour. The saturation point D_{sat} is the loss when every tile takes exit j, the shallowest the split depth allows. Neither is a quantity to search for. Each is one forward pass per rate with the exit map held constant, which is what makes the rescaling in Section E.4 cheap enough to be worth doing in deployment.

Between them the saving is fixed by the ladder rather than by the budget. With c_k the cost of exit k in units of one released full-frame decode, j the split depth and K the number of exits, the ceiling is closed form under either compute denominator:

$$S_{\max}^{\text{rel}} = 100(1 - c_j), \quad S_{\max}^{\text{self}} = 100(1 - c_j/c_{K-1}) \quad (58)$$

With $c_j = 0.6088$, recorded in results/saturation_RECIPE512_ctc53.json, and $c_{K-1} = 1.0095$, recorded as deepest_exit_cost in results/router_RECIPE512_b01.json, the first reads 39.1% and the second 39.70%. The deepest exit costs more than one released decode because it carries the seam repair, which the released decoder does not run, and that difference is the whole of the gap between the two ceilings.

Section D tabulates both ends at all nine measured rates, and that table is not repeated here. The one property of it this section needs is that a budget in decibels means something different at each rate: both ends widen as the rate falls, the band widens faster than the floor rises, and the 0.1 dB budget that sits 45% of the way up the band at q0 sits 9% of the way up at q63. The rescaling of Section E.4 turns out to need that position rather than the decibel figure.

A budget below the floor admits no allocation at all, which is a different failure from a budget falling in a gap of the Lagrangian hull, where the sweep returns the nearest reachable point and Everett's argument still makes it optimal for the compute it consumes [28, 29]. Below the floor the feasible set is empty and there is nothing to return. Three cells of the grid in Section E.3 are in that state and are recorded as such rather than dropped: at a 0.05 dB budget the three highest rates, q32, q48 and q63, have floors of 0.053, 0.060 and 0.066 dB.

Three estimates of the floor are in play and they are not interchangeable. Table 122 gives all three, because the rescaling below divides by the distance between the floor and the saturation point, and near the floor that is a small difference divided by a small difference.

source	what it measures	q0	q63
the saturation file	a deployed decode with every tile at full depth, over 53 frames	0.036	0.072
the dense grid	the same quantity, over 106 frames	0.033	0.066
the second sweep	the per-tile table at a zero multiplier, where a tile takes its own lowest-error exit	-	0.066

Table 122. Three measurements of the floor, in decibels. The first two differ by three to six thousandths of a decibel because they are the same quantity over a different number of frames. The third is a different quantity: a tile whose shallower exit happens to have the lower error takes it, so the table's floor sits below the deployed one. Everything downstream of here takes both ends of the window from the first row, so that the floor and the saturation point are on one measurement.

The three rows are results/saturation_RECIPES12_ctc53.json, results/signalled_RECIPES12_grid.json and results/supp_paper_curve_PAPER.json, all on the pinned checkpoint and all with decibels averaged per frame. The third records a floor only where it found the 0.05 dB budget unreachable, which is why q0 is empty.

E.2. Three conventions, and the conversions between them

Three separate choices have to be pinned before a figure in this section means anything, and the project makes all three in two ways in different files. The loss can be measured against the released decoder's full-frame decode of the same latent, which puts the cost of cutting the frame into tiles inside the budget, or against this model's own deepest exit decoded tile by tile, which cancels it. Decibels can be averaged per frame and then over frames, which is what the released codec's own evaluation does, or pooled into one mean squared error first. And the saving can be divided by the released decode or by our own deepest exit. Two of the three conversions are exact:

$$D^{\text{rel}} = D^{\text{self}} + D_{\min}, \quad S^{\text{rel}} = 100 - c_{K-1} (100 - S^{\text{self}}) \quad (59)$$

choice	the two options	what separates them
loss measured against	the released full-frame decode, or our own tiled deepest exit	the floor: 0.036 dB at q0, 0.072 dB at q63
decibels averaged	per frame and then over frames, or pooled over every tile first	measured, not closed form: a pooled 0.1 dB is 0.117 dB per frame at q0 and 0.122 at q63
saving divided by	the released decode, or our deepest exit at 1.0095 of it	a self-referenced 30% is 29.33% release-referenced, and a self-referenced 0% is -0.95%

Table 123. What has to be stated with every decibel and every saving in this section. Take from it that a number quoted without its convention is uncertain by about a third of a 0.1 dB budget on the quality axis and by about two thirds of a point on the compute axis. The first and third rows convert exactly, by the identity above; the second does not, because a mean of logarithms is not the logarithm of a mean.

The floor is results/saturation_RECIPES12_ctc53.json; the two averaging figures are the 0.1 dB operating points of results/supp_paper_curve_PAPER.json, which bisects on the pooled decibel and records the per-frame decibel of the same allocation beside it; the deepest-exit cost is deepest_exit_cost in results/router_RECIPES12_b01.json. The averaging convention itself is defined in Section A.

E.3. The grid

Table 124 is the sweep the main paper quotes: nine budgets across five rates, measured against the released decoder on both axes, with decibels averaged per frame, two frames from each of the 53 test sequences. Allocation is by the exact per-tile losses, so every entry is an upper bound on what a decoder-side router can reach.

budget dB	q0	q16	q32	q48	q63
0.050	16.4	9.3	none	none	none
0.075	24.3	19.7	14.6	11.1	7.6
0.100	29.8	25.5	20.7	18.2	15.6
0.150	36.6	33.0	28.6	26.5	24.1
0.200	39.1	37.8	33.7	31.6	29.6
0.250	39.1	39.1	37.4	35.2	33.0
0.300	39.1	39.1	39.1	37.9	35.8
0.400	39.1	39.1	39.1	39.1	39.1
0.500	39.1	39.1	39.1	39.1	39.1

Table 124. Compute saved against budget, release-referenced. *none* marks a budget below that rate's floor, where no allocation is admissible. Reading down a column, the return on a looser budget falls away long before the ceiling is reached. Reading across a row, the rate dependence is largest at the tightest budget and vanishes once every rate has saturated. The 0.1 dB row reproduces the headline of the main paper, 29.9% at q0 and 15.6% at q63, to within a tenth of a point on a different frame count.

results/signalled_RECIPES12_grid.json, pinned checkpoint, two frames per sequence, budget bisected on the delivered per-frame decibel of a real decode of the mixed map. The headline file results/signalled_RECIPES12_ctc53.json uses one frame per sequence and reports 29.9% and 15.6% at the same budget.

A second sweep of the same checkpoint prices the same window differently, and Section D prints its operating points in full. It bisects on the pooled decibel rather than the per-frame one and reads 53 sequences at one frame each rather than two, and at a nominal 0.1 dB it saves several points more at every rate than the grid above. The conversions of Section E.2 account for part of that and not for all of it, which is the first sign of what Section E.5 measures. Its own saturation points are the useful thing here: the budget at which a rate runs out of allocation is 0.20 dB at q0, 0.25 dB at q16, 0.30 dB at q32, 0.50 dB at q48, 0.50 dB at q63. The three budgets the paper reports are therefore not three points on one curve. At 0.1 dB every rate still has an allocation to make, at 0.3 dB only q48 and q63 do, and at 0.5 dB none does.

results/supp_paper_curve_PAPER.json, pinned checkpoint, 53 sequences at one frame each, 35 operating points at seven budgets. A saturated point is flagged in the file rather than inferred here, and carries no exit histogram, because the allocation has stopped changing.

Budgets in [0.179, 0.194] dB saturate the lowest rate and no other, a window 15 thousandths of a decibel wide. Above 0.396 dB every rate has saturated, so the binding constraint is the ladder rather than the budget. Both figures are read off the nine-rate table in Section D rather than off either sweep.

E.4. Rescaling onto the band

With both ends of the window measured, a budget can be quoted as a position in the window rather than as a decibel figure. Write

$$u = \frac{D - D_{\min}}{D_{\text{sat}} - D_{\min}} \quad (60)$$

so that $u = 0$ is the floor and $u = 1$ the saturation point. At a matched decibel the five rates are 15.5 points apart in saving. At a matched u they are 1.7 points apart on average and 2.2 at worst, so the window accounts for most of the rate dependence of the trade-off and a couple of points are left over.

u	q0	q16	q32	q48	q63	spread
0.1	16.5	14.6	15.1	15.2	16.3	1.91
0.2	21.0	20.9	20.7	20.9	21.8	1.13
0.3	25.2	24.9	24.3	25.5	26.2	1.91
0.4	28.3	27.7	27.9	28.6	29.7	1.96
0.5	30.8	30.4	30.4	31.4	31.9	1.54
0.6	32.8	33.0	32.8	33.4	33.9	1.13
0.7	34.7	34.7	34.7	35.3	35.7	1.00

Table 125. The five rates at matched positions in their own windows. Read across a row: five rates whose raw savings at a matched decibel differ by 15.5 points agree here to about a point and a half. The spread is largest at the bottom of the window, where the floor subtraction is a large fraction of a small number and the two files disagree about the floor by a few thousandths of a decibel.

Computed in the build from results/signalled_RECIPES12_grid.json and results/saturation_RECIPES12_ctc53.json by the rescaling in scripts/tradeoff_figure.py, linearly interpolated between measured points. Every rate has measured points spanning $u = 0.10$ to 0.70 , so no entry is an extrapolation. Mean spread over these seven positions 1.51 points, worst 1.96, against the 1.7 and 2.2 recorded in results/band_collapse.json, which interpolates on a wider grid and holds the last measured value beyond each rate's last point.

The collapsed curve is then described by one free number. With C the architectural ceiling, which is not fitted,

$$S(u) \approx Cu^\beta \quad (61)$$

and fitting $\ln(S/C)$ against $\ln u$ by least squares through the origin gives 0.38 on the pinned checkpoint, with $R^2 = 0.989$ and a worst residual of 2.0 points. The same procedure on the second training run gives 0.33 with $R^2 = 0.984$, which is the replication Figure 51 draws and Section E.5 takes apart.

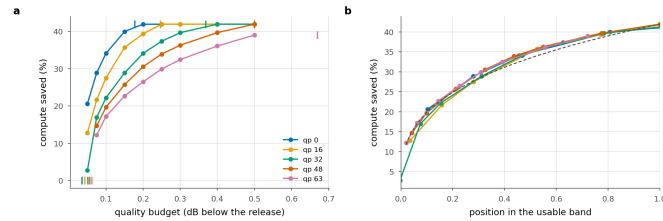


Figure 51. The collapse on the second training run. **a** Saving against budget, one line per rate, with each rate's floor and saturation point ticked. **b** The same with the budget axis rescaled onto each rate's own window; dashed is the one-parameter power law. The main paper shows this pair for the pinned checkpoint.

docs/figures/budget_band_BEST.png, drawn by scripts/tradeoff_figure.py from results/signalled_BEST_grid.json and results/saturation_BEST_ctc53.json, both on runs/BEST/ckpt_eval.pth.tar rather than on the pinned checkpoint.

E.5. How weak the replication is

The main paper says the collapse replicates and the exponent does not. Two checkpoints is a thin basis for either half of that sentence, and Table 126 is meant to show how thin. Each row refits the same one-parameter law after changing exactly one thing, over the window of positions where every sweep has points.

what is changed	β before	β after	Δ
the checkpoint, pinned to BEST, each with its own ceiling	0.360	0.310	-0.050
the sweep, 106 frames to 53, one checkpoint	0.360	0.261	-0.098
the budget axis, per-frame decibels to pooled, one sweep	0.272	0.180	-0.093
the assumed ceiling C , 39.1 to 41.91	0.360	0.421	0.061
the floor, the saturation file to the grid's own	0.360	0.369	0.010
the rate, fitted one at a time, q63 against q32	0.321	0.385	0.064
the fit form, $1/b$ from the inverse fit, per rate	0.326	0.459	0.133

Table 126. What moves the exponent. Take from it that the exponent describes a sweep rather than a decoder. Every row except the floor moves it by as much as changing the checkpoint does, re-sweeping the same checkpoint moves it twice as far, and reading the budget axis in the other averaging convention moves it almost as far again. The comparison the main paper draws between 0.38 and 0.33 is inside the noise of every other choice on this list.

Computed in the build. Rows 4, 5 and 6, and the before column of rows 1 and 2, refit results/signalled_RECIPES12_grid.json against results/saturation_RECIPES12_ctc53.json; the after column of row 1 uses results/signalled_BEST_grid.json and results/saturation_BEST_ctc53.json on runs/BEST/ckpt_eval.pth.tar, whose ceiling is its own; the after column of row 2 uses the operating points of results/supp_paper_curve_PAPER.json and row 3 uses its raw sweep; row 7 is results/frontier_low.json, fitted to results/curve_BEST.json on the same second run over 40 sequences. Every fit is over u in $[0.15, 0.85]$ and through the origin in log space, on 16 to 22 points. R^2 stays between 0.911 and 0.980 across the rows, so no row is a bad fit to its own data.

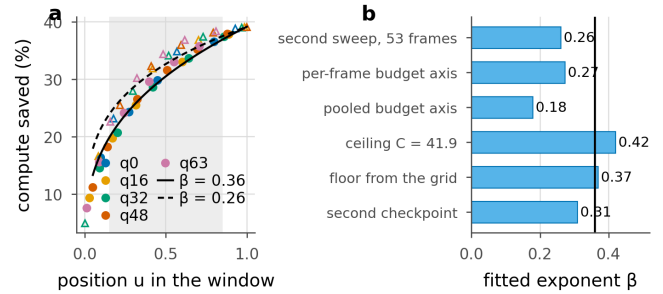


Figure 52. The collapse, and what the exponent is sensitive to. **a** The rescaled points from two sweeps of the pinned checkpoint, filled circles for the 106-frame grid and open triangles for the 53-frame frontier, with the power law fitted to each over the shaded window. The two sweeps of one checkpoint do not lie on one curve. **b** The exponent under one change at a time, with the vertical line at the exponent of the first row of Table 126. The change of checkpoint, which the main paper reads as a finding, is the smallest of the six.

docs/figures/band_sensitivity_PAPER.png, drawn by scripts/supp_fig_band_sensitivity.py, which imports the fit from this section's module so that the figure and the table cannot disagree. Sources as for the table above.

The floor is the one input the exponent is insensitive to over this window, and only over this window. Refitting over the whole range instead, where the points below $u = 0.15$ sit, moves it from 0.379 to 0.417 when the floor is taken from the grid rather than from the saturation file. A rescaling is only as good as the two numbers it divides by, and near the floor it is dividing a small difference by a small difference.

One stored file needs regenerating before submission. Re-running the fit on the inputs results/band_collapse.json names, as they stand now and over the full range it used, gives $\beta = 0.3792$ with $R^2 = 0.9895$, against the 0.3767 and 0.9890 the file records; both of its inputs were rewritten after it was written, and it still carries the pre-correction ceiling of 41.91. The two digits the paper quotes, 0.38 and 0.989, survive the regeneration unchanged, so nothing in the main text moves. results/band_collapse_BEST.json reproduces exactly from its inputs: 0.3253 and 0.9843 against the 0.3253 and 0.9843 recorded.

E.6. What the operating window does not pin

- The floor and the saturation point are measured, cheap and unambiguous. Each is one forward pass per rate, and the ceiling between them is closed form in the per-exit costs.
- The collapse is a strong empirical regularity on both checkpoints. Rescaling takes the spread across rates from 15.5 points to under two at matched positions.
- The exponent is a description of one sweep. It moves further when the same checkpoint is swept again, or when the budget axis is read in the other averaging convention, than it moves between the two training runs the main paper compares.
- Everything here is the oracle allocation, obtained with the per-tile losses known. It bounds any router and says nothing about how much of the window a decoder-side predictor can reach.
- Every entry is a mean over the test set at one global multiplier. The spread behind those means is in Section D; no configuration was trained twice, so neither section bounds the variation between two runs of the same recipe.
- The window is measured with the exit map chosen by an exact multiplier sweep. Nothing here says how wide the window looks to a decoder that has to guess the map, which is what Section F measures.
- The grids are intra frames at 1080p and below, on the CTC set. Nothing here is measured on inter frames, and the window has not been measured above 1080p, where the evaluation card does not have the memory for a decode.

F. The router and the parameter-free rule

An exit map assigns one exit index to every tile of a frame, and this work produces one in three ways. Configuration A searches all K exits for every tile at the encoder, which holds the source, and signals the answer. Configuration B predicts the map at the decoder from data the decoder already holds, and signals nothing. The third way computes it at the decoder from one number the entropy coder has already produced, with nothing learned. The head architecture, the inputs it reads and the rule by which it turns logits into an exit are given in the main paper's allocation subsection, and none of that is repeated here. What is here is what those inputs are worth, what the head costs to run, and how it fares against the third way.

Write $D(t,k)$ for the mean squared error of tile t decoded at exit k and c_k for the cost of exit k in units of one released decode. At a price λ on compute, the Lagrangian oracle gives each tile the exit that minimises

$$\ell(t,k) = D(t,k) + \lambda c_k, \quad k^*(t) = \arg \min_{k \geq j} \ell(t,k) \quad (62)$$

The split depth j is the number of trunk groups every tile runs before any tile may leave, so exits below j do not exist and the minimisation starts there. Throughout this section, *agreement* means the fraction of tiles on which a predictor picks the same exit as this oracle at the same λ . It is the quantity the head is trained on. Part of what F.4 reports is that it is not the quantity that decides which allocation is cheaper.

F.1. What each input is worth

The head reads four per-tile signals and one per-frame signal. Since F.4 finds that a rule reading a single number beats it, it is fair to ask which of the five carries anything. The ablation trains the same head once per input group, with every other group zeroed after its projection rather than deleted. Architecture, parameter count, optimiser, seed, image order and quality draws are then identical across the six runs, and the only thing that differs is how much the head is allowed to know. All six carry 148,176 parameters, which is the deployed head's 144,024 plus the projection for the bit count that the deployed head does not use.

The quality index q stays live in every single-group run. It is one number per frame, identical for every tile in that frame, so it can say which operating point the decoder is at and cannot, even in principle, tell two

tiles of one frame apart. Holding it live keeps the runs comparable on per-tile information alone. The run with q on its own is then the floor that choice implies, which is the agreement reachable with no per-tile information whatever.

live inputs	agreement	$\pm$ s.e.	last 250	over floor
stem,q	0.7750	0.0074	0.7607	+0.2744
stem,latent,scales,bits,q	0.7721	0.0075	0.7543	+0.2715
latent,q	0.7448	0.0080	0.7433	+0.2442
scales,q	0.7156	0.0083	0.7170	+0.2150
bits,q	0.6202	0.0091	0.6137	+0.1196
q	0.4923	0.0114	0.4900	-0.0083

Table 127. What each router input is worth. Read the first column against the last: only the stem moves agreement far from what a single constant exit already achieves. Agreement with the oracle's exit choice on 1,200 frames and 4,800 tiles the head never trained on, with one standard error taken across frames rather than across tiles, since tiles of one image are not independent draws. *last 250* is the running held-out agreement over the final 250 training steps, on 12 tiles a step, and confirms that the end-of-run measurement is not a fluctuation. *over floor* is the margin over the best single constant exit, which scores 0.5006 on these tiles.

results/router_ablation.json, and the six per-variant training records results/router_ablation_stem.json and its five siblings. All on the pinned checkpoint, all at $\lambda = 1.3 \times 10^{-5}$, all 3,000 steps from seed 0.

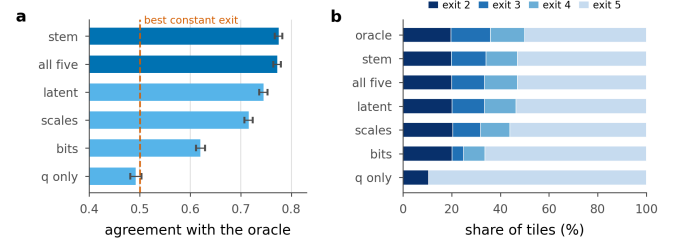


Figure 53. The input ablation, both halves of it. **a**, agreement with the oracle per variant, with one standard error across frames; the dashed line is the best single constant exit. The two dark bars are the two heads that see the stem. **b**, where each variant sends its tiles, against the oracle's own distribution. Every variant reproduces the oracle's use of the shallowest exit to within a point, and what degrades as inputs are removed is the separation of the middle of the ladder: the bit count alone nearly empties exit 3, and q alone uses two of the four exits available to it.

Drawn by scripts/supp_router_figs.py from results/router_ablation.json, fields agree, stderr, pred_hist and oracle_hist.

The stem alone. Reading the stem and q gives 0.7750; reading the stem, the latent, the scales, the bit count and q gives 0.7721. The difference is 0.0029 against a standard error of 0.0105 on the difference, so it is under a third of one standard error and its sign is not determined. Four of the five inputs add nothing this measurement can resolve. That result is about those four inputs in this head, and it does not show that the latent, the scales or the bit count are uninformative about a tile. What it shows is that whatever they carry is already carried by the stem, which is computed downstream of all of them.

The single-group ordering. The stem at 0.7750 is 0.0302 above the latent, against a standard error of 0.0109 on the difference; the latent is 0.0292 above the scales; and the scales are 0.0954 above the bit count, which is more than seven standard errors. The bit count is the weakest per-tile input the head has, and F.4 shows a rule that reads the bit count and nothing else beating the head at every rate. The input the head learns least from is the input that wins when it is used differently.

The floor. The quality index on its own reaches 0.4923 ± 0.0114 where the best single constant exit scores 0.5006. A head with no per-tile information cannot beat the best constant in expectation, and this one does not. The floor belongs beside every other row, because an agreement of 0.62 cannot be read at all until it is known that 0.50 is free.

The ablation's own cost. The six trainings took 6.5 hours of one NVIDIA RTX A6000 between them, from 62 to 66 minutes each

(results/router_ablation_*.json, field wall_s). The decoder is frozen for all of them, so none of that is decoder training. It is the price of the question, and it is worth stating beside a negative answer.

F.2. What the head costs

q	decode (ms)	stem (ms)	router (ms)	share (%)
0	114.29	41.25	0.571	0.500
32	114.41	41.32	0.568	0.497
63	114.44	41.31	0.568	0.496

Table 128. The head against the decode it decides for. The share column is the number to take away, and it is three times what the operation count predicts. Three timings on one latent, interleaved so that a co-tenant's load drift lands on all of them equally. *decode* is the tiled decoder at the deepest exit, which is what the head's share is a share of; *stem* is the first j groups, which the head does not pay for because the decoder runs them anyway before the split. The head takes 0.50% of the decode where its operation count predicts 0.163%, a factor of 3.0.

results/router_latency.json: 1920×1080 padded to 2048×1280, 40 iterations after warm-up, one NVIDIA RTX A6000, the 144,024 parameter head runs/RECIPE512/routers2/v2_lam1.3e-5.pth. Measured on runs/RECIPE512/ckpt_eval.pth.tar rather than on the pinned checkpoint; the quantity is a ratio of two timings of the same weights, so it does not depend on which epoch they came from.

The extra 0.33 points are launch overhead. A head of 144,024 parameters evaluated on 40 vectors is small enough that its wall-clock is dominated by the cost of starting its kernels, and an operation count does not contain that. Every configuration-B saving in this work charges the head at its operation share, which is the smaller of the two numbers and therefore the more flattering to us; the honest reading is that the head costs about half a percent of the decode in time. It stays small against what it decides about, since one trunk block is 7.5% of the decode by the cost vector below.

Two heads, two shares. The pinned checkpoint carries a head that was trained jointly with the decoder, and a second head was trained afterwards against that decoder once it was frozen. They are different objects and they are priced differently: the jointly trained one is charged 0.044% of the decode and the frozen-decoder one 0.163%, because the second reads the decoded latent and the entropy model's scales as well as the stem. Both shares are measured by hooks on the head's own convolutions and linear layers rather than assumed, in the same run that produced the saving. F.4 reports both heads.

results/router_RECIPE512_b01_joinhead.json and results/router_RECIPE512_b01.json, field router_compute_share_pct in each.

For scale, what the search costs instead. Configuration A does not run a head; it runs the argmin itself, which needs the true error of every exit. On the pinned checkpoint at q63 one tiled 1080p decode takes 115 ms, the deployed table costs 4.52 of those and the full-frame table 1.37. A decoder-side head at half a percent of one decode, and a rule at nothing, are both several orders below that, which is the whole reason for wanting one.

results/supp_encoder_cost_PAPER.json, pinned checkpoint, 15 iterations on one NVIDIA RTX A6000.

F.3. The calibrated bit rule

The entropy model produces one number per tile before the trunk runs, at no cost, and then discards it: how many bits that tile's latents took. Per-block bit allocation is a standard quantity in learned compression, where it is something to choose, and block-level rate control sets it so that complex regions get more bits [42]. Here it is read in the other direction, after the fact and at the decoder, as a statement about how hard the region was. The results files call the rule *rate-rank*; the paper calls it the calibrated bit rule. What follows is all of it.

The statistic. Sum the entropy coder's estimated bits r_i over the latent positions i falling inside tile t , and divide by the mean over the N tiles of that frame. Normalising per frame rather than globally is deliberate: the absolute rate level is a property of the frame, and what decides a tile is

how it compares with the rest of its own frame.

$$b(t) = N \frac{\sum_i r_i}{\sum_i r_i} \quad (63)$$

The surrogate. Assume every tile has the same shape of decay across the ladder, up to a scale that depends only on $b(t)$.

$$\log D(t, k) \approx \alpha \log b(t) + c + \log \varphi_k \quad (64)$$

ϕ has one entry per reachable exit and says what that exit costs on an average tile; α is an exponent fitted rather than assumed. Fitting it is what lets the rule find the direction of the relation instead of asserting one, and an earlier version of this surrogate that fixed the exponent at 1 was wrong about that direction. The fit is offline and leave-one-sequence-out, so no sequence contributes to the profile that routes it, and its whole output is six numbers per rate.

The decision. Run the oracle's own Lagrangian on the surrogate table in place of the true one, and bisect λ against the budget as configuration A does.

$$\hat{k}(t) = \arg \min_{k \geq j} [e^c b(t)^\alpha \varphi_k + \lambda c_k] \quad (65)$$

the decoder, once per frame

- 1 *input* r , the entropy coder's estimated bits per latent position; the cost vector c ; the price λ ; the calibration (α, c_0, ϕ) for this quality index
- 2 *for* each tile t of the N tiles of the frame
- 3 $b \leftarrow N \cdot (\text{sum of } r_i \text{ inside } t) / (\text{sum of } r_i \text{ over the frame})$
- 4 *for* $k = j \dots K-1$
- 5 $L_k \leftarrow e^{c_0} \cdot b^\alpha \cdot \phi_k + \lambda \cdot c_k$
- 6 $\text{exit}(t) \leftarrow$ the k that minimises L_k
- 7 *return* exit

offline, once per quality index, leave-one-sequence-out

- 8 $d(t) \leftarrow$ mean over $k \geq j$ of $\log D(t, k)$
- 9 $(\alpha, c_0) \leftarrow$ least squares of $d(t)$ on $\log b(t)$
- 10 $\log \phi_k \leftarrow$ mean over t of $[\log D(t, k) - \alpha \log b(t) - c_0]$

Table 129. The calibrated bit rule, in full. Lines 1 to 7 are what the decoder runs: one power and K products per tile, no parameters, nothing signalled. Lines 8 to 10 are the offline calibration, whose entire output is the six numbers per rate printed in the next table. λ is bisected in two levels, on the cheap per-tile table first and then against a real decode of the resulting map, because the table and the decode differ slightly at the tile seams. Sweeping λ traces the lower convex hull of the achievable set in the sense of [28], so the point returned is the best one on that hull at the budget it consumes.

q	delivered dB	α	ϕ_2	ϕ_3	ϕ_4	ϕ_5
0	0.0999	9.98	1.0242	1.0009	0.9912	0.9841
16	0.1002	4.58	1.0251	1.0021	0.9911	0.9822
32	0.1000	2.73	1.0297	1.0013	0.9893	0.9804
48	0.1000	1.94	1.0350	1.0004	0.9884	0.9771
63	0.0999	1.43	1.0494	0.9963	0.9829	0.9731

Table 130. The rule's entire state, at the 0.1 dB budget. There is nothing else to store, and it is printed here so that the rule can be reproduced without refitting it. α is positive everywhere, so a tile whose latents cost more bits is modelled as harder at every exit and is sent deeper, which is what the correlations in F.4 confirm. ϕ is nearly flat, spanning 4.1% at q0 and 7.8% at q63 between its shallowest and deepest entry, against costs that span 0.61 to 1.01, so what moves a tile along the ladder is its own $b(t)$ rather than the shape of the profile. *delivered dB* is what the two-level bisection actually landed on.

results/raterank_RECIPE512_b01.json, pinned checkpoint, 53 CTC sequences at one frame each.

The costs c_k on the pinned checkpoint are 0.6088, 0.7578, 0.8697, 1.0095 for exits 2 to 5, recovered from the frame-level saving of maps that send every tile to one fixed exit. The shallowest of them, $c_j = 0.6088$, sets the architectural ceiling of 39.1%. The deepest is above 1 because our ladder carries seam repair and an adapter that the released

decoder does not.

results/static_RECIPES12_b01.json, field uniform, and
results/saturation_RECIPES12_ctc53.json, fields cost_j and ceiling_pct.

F.4. The rule against the trained head

Two trained heads exist on this checkpoint and both are reported, since averaging them would hide the only spread this work has. *joint* is the head the checkpoint carries, trained alongside the decoder and read straight out of it. *frozen* is the 144 K head trained afterwards, against the decoder once it had stopped moving. Both were evaluated on the pinned checkpoint over 53 sequences at one frame each, by one script, at the same budgets, and both are charged for their own compute.

q	0.1 rule	0.1 joint	0.1 frozen	0.3 rule	0.3 joint	0.3 frozen
0	27.79	26.84	24.41	39.12	39.08	38.96
16	22.72	18.36	20.95	39.12	39.08	38.96
32	18.24	13.21	17.43	39.12	39.08	38.96
48	14.67	8.26	14.15	37.73	37.03	33.40
63	11.99	5.13	11.14	34.34	34.07	27.73

Table 131. The parameter-free rule against two trained heads, as the percentage of the released decoder's operations saved, at two budgets and five rates. The rule is ahead of both heads in every cell of this table. Every column is charged for what it costs: each head pays its own compute share and the rule pays nothing. The two heads are further apart from each other than either is from the rule at several rates, which F.5 takes up.

Rule: results/raterank_RECIPES12_b01.json and b03. Joint head: results/router_RECIPES12_b01_jointhead.json and its b03 sibling. Frozen head: results/router_RECIPES12_b01.json and b03. All on runs/RECIPES12/ckpt_PAPER.pth.tar.

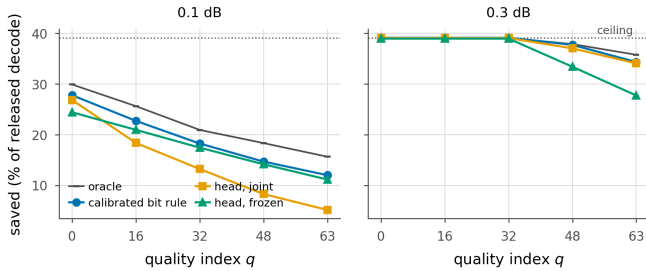


Figure 54. The rule sits between the two heads and the oracle. Saving at each rate, against the Lagrangian oracle measured inside the same run and the architectural ceiling (dotted). At 0.1 dB the rule is above both heads at every rate, and the gap between the two heads is wider than the gap between the rule and either of them. At 0.3 dB the three lowest rates saturate and the three curves separate only at q48 and q63.

Drawn by scripts/supp_router_figs.py from the six files named in the table above and results/saturation_RECIPES12_ctc53.json.

At the working budget. Against the joint head the rule is ahead by +0.95 points at its narrowest and +6.87 at its widest, the latter at q63. Against the frozen head it is ahead by +0.53 to +3.38 points. The rule reaches 27.8% at q0 and 12.0% at q63, and it is ahead at all 5 measured rates whichever head it is measured against. A head trained on this decoder against this oracle returns nothing over a rule with no parameters, while carrying a compute share that the rule does not.

At 0.3 dB the low rates saturate and the arithmetic is exact. At q0, q16 and q32 all three configurations send every tile to the shallowest exit, so the maps are identical and the only difference left is the price of deciding. The joint head falls 0.044 points below the 39.1% ceiling and the frozen head 0.163, which are their compute shares of 0.044% and 0.163% to three decimals. At q48 and q63 the budget still binds and the allocations separate: the rule reaches 37.7% and 34.3% against the joint head's 37.0% and 34.1% and the frozen head's 33.4% and 27.7%. A looser budget gives an allocation more room to be wrong in as well as more room to be right, and the frozen head uses it to be wrong.

Why 0.5 dB is not in that table. At 0.5 dB every rate saturates for every configuration, so all three maps are identical and the only thing a comparison could measure is the cost vector each file was written with. The rule's 0.5 dB file is on runs/RECIPES12/ckpt_eval.pth.tar, where the shallowest exit was priced at 0.5809 against the pinned 0.6088, a difference worth 2.79 points of apparent saving. The file is results/raterank_RECIPES12_b05.json and it records that the rule matches the oracle map exactly at all five rates there, which is the only thing read from it.

q	rule agrees	p depth	p level	p spread	oracle
0	0.458	+0.54	+0.83	+0.61	29.90
16	0.319	+0.52	+0.85	+0.66	25.62
32	0.280	+0.53	+0.86	+0.71	20.93
48	0.350	+0.53	+0.84	+0.71	18.33
63	0.390	+0.38	+0.88	+0.68	15.64

Table 132. Why the rule works, and it is not by agreeing, at 0.1 dB. Compare the second column with the fifth: the rule picks the oracle's exit on a minority of tiles and still tracks the oracle's saving. The three Spearman correlations are between a tile's bit count and, respectively, the depth the oracle assigns it, its mean distortion across the ladder, and the spread between its shallowest and its deepest exit. *oracle* is the Lagrangian oracle measured inside the same run, which is the ceiling the rule is chasing.

results/raterank_RECIPES12_b01.json. The file stores spearman_bits_vs_exit against the negated exit index, as scripts/raterank_curve.py computes it, so the correlation with depth quoted here is its negative. The oracle column is measured at one frame per sequence and is not charged for the exit map; deployed configuration A, which carries the map, reads 29.9, 25.6, 20.9, 18.3, 15.6% over the same five rates at two frames per sequence (results/signalled_RECIPES12_ctc53.json).

The rule agrees with the oracle on 0.28 to 0.46 of tiles, well under the frozen head's held-out 0.718, and saves more at every rate. Agreement counts a disagreement between two exits that are within a hair of each other exactly as heavily as one that costs most of the frame's error, and most disagreements are of the first kind. What the rule gets right is the ordering. A tile's bit count correlates with the spread across the ladder, which is how much that tile stands to gain from depth, at 0.61 to 0.71 at every rate, and with the depth the oracle actually assigns at +0.38 to +0.54.

What the rule cannot do is see past that ordering. A rank-1 model gives every tile the same relative profile over exits, so b(t) decides where on the ladder a tile falls and never the shape of its trade-off. That is the ceiling this baseline sits at, and it is the part a learned head would have to earn its parameters on. Neither of our two heads does.

F.5. How much a trained head varies

No head in this work was trained twice at two seeds, so there is no seed variance to report and none was measured; results/router_ablation.json records seed 0 for all six of its runs and one_checkpoint true. A second training run is a training run, and the evaluation queue that produced the rest of this supplement was restricted to inference. What can be reported is the spread between heads that were trained independently of each other, which is a looser quantity than seed variance and a larger one.

Two heads on one decoder. The joint and frozen heads differ by +2.43 points at q0 and by -6.01 at q63, a range of 8.44 points across the five rates at 0.1 dB, and they cross: the joint head is the better of the two at q0 and the worse at every other rate. At 5 of the five rates the two heads differ from each other by more than the rule's margin over whichever of them is nearer, so a comparison against one trained head says less than it appears to. That is the reason both are printed in F.4 rather than the better of them.

q	oracle	head	rule	margin	rule agrees
0	34.02	31.56	30.92	-0.64	0.479
16	27.67	25.07	27.14	+2.07	0.370
32	22.29	16.47	22.39	+5.92	0.399
48	19.75	13.27	19.10	+5.83	0.431
63	17.19	8.99	16.16	+7.17	0.479

Table 133. The same comparison on a second training run. The margin column is the point: it is positive at four of the five rates and wider than on the pinned run, so the pinned numbers are the conservative ones. BEST is a separate recipe at a different epoch with its own head, trained the same way. The rule is ahead at 4 of 5 rates, by up to 7.2 points, and stays within 3.1 points of the oracle everywhere. The one rate it concedes is the only rate on either run where a trained head finishes ahead, and it does so by under a point.

results/raterank_BEST_compare.json: the BEST run, 53 sequences, 0.1 dB, not the pinned checkpoint.

q	before	after	change	β before	β after
0	27.18	30.27	+3.10	136.6	19.0
16	23.27	25.82	+2.54	88.8	7.7
32	19.40	19.43	+0.03	9.1	-19.1
48	16.06	15.54	-0.51	-39.7	-35.9
63	12.94	9.58	-3.36	-50.3	-42.2

Table 134. Agreement rises and the saving does not follow. The change column has both signs. The head was retrained after the exit mask was fixed, on the same recipe at the same λ ; held-out agreement went from 0.718 to 0.808 while the deployed saving moved by +3.1 points at q0 and -3.4 at q63. β is the cost multiplier the bisection applies to move a head trained at one λ onto this operating point, and the retrained head is ahead where that multiplier is small and behind where it is large. This is the same lesson as the rule's low agreement, from the other direction.

results/router_retrain_compare.json. The file records no checkpoint, and its before column reads 27.18% at q0 where the pinned measurement in results/router_RECIPES12_b01.json reads 24.41%, so the two are not on one basis. Only the comparison of the two heads within the file is read here.

F.6. What this section does not measure

- No seed variance, for either head or for any ablation variant. Every run in results/router_ablation.json is seed 0 on one checkpoint, and nothing here says the ordering of the input groups would survive a second seed or a second decoder.
- The ablation reports agreement and nothing else. No variant was carried through a budget bisection to a deployed saving, so this section cannot say what a stem-only head would save, and the argument that agreement and saving come apart applies to the ablation as much as to the head.
- The frozen-decoder head was trained against one checkpoint and evaluated on another. It was trained against runs/RECIPES12/ckpt_eval.pth.tar, which the watchers overwrite, and then run on the pinned checkpoint (results/router_RECIPES12_b01.json, field router2_meta), while the ablation heads were trained against the pinned checkpoint itself. Its 0.718 and the ablation's 0.7721 are two measurements of different things.
- One head covers all five rates. A head per rate is the obvious remedy for the cost multiplier in the retrain table and is not measured here.
- The head is timed at batch 1 on one GPU class, and its share of the decode in seconds is three times its share in operations. What that factor would be on a device where kernel launches are cheaper is not measured.
- The rule's 0.5 dB file is off the pinned checkpoint, which is why that budget is described rather than tabulated.